CIS 678 - Machine Learning

– Clustering–

Instructor: Yong Zhuang

yong.zhuang@gvsu.edu

Last Class

- Artificial Neural Networks (ANN)
  - Architecture
  - Activation function
  - Loss function
  - Optimization
  - Regularization
  - Training
    - Stochastic gradient descent + chain rule
    - Backpropagation

Agenda

- Introduction to Clustering
- K-Means
  - K-Means Algorithm
  - Limitation of K-Means
  - K-Means Implementation
- Agglomerative Clustering
- Self-Organizing Maps (SOM)

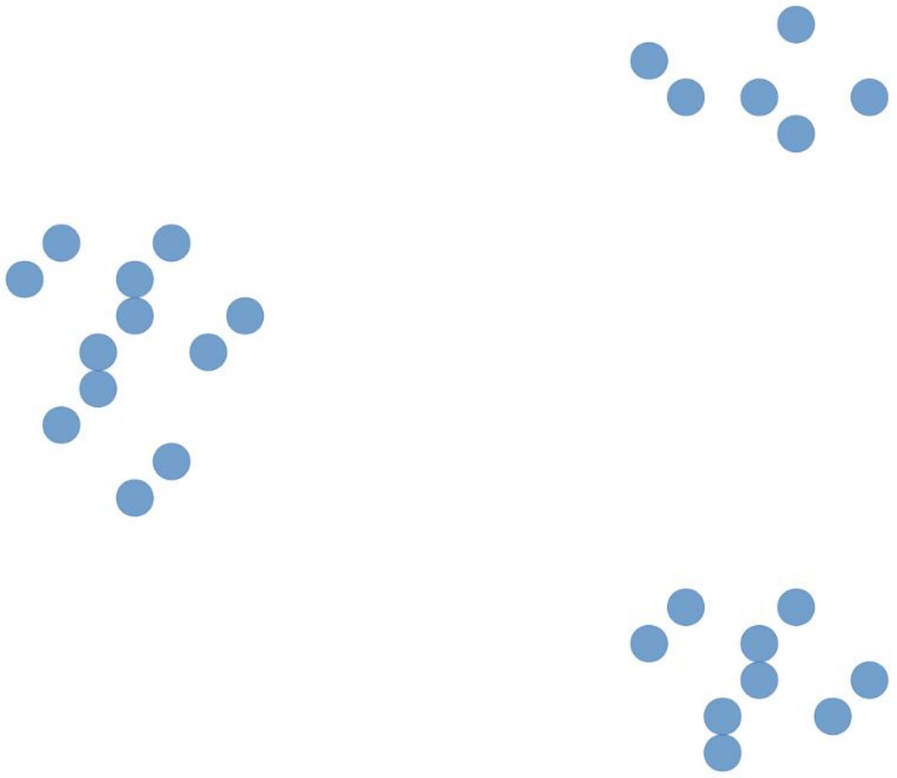
Clustering

- Clustering is the task of discovering unknown subgroups in data, or clusters
- The goal is to partition the dataset into clusters where ‘similar’ items are in the same cluster and ‘dissimilar’ items are in different clusters
- Example:
  - Social Network Analysis: Clustering can be used to find communities
  - Ecology: cluster organisms that share attributes into species, genus, etc…
  - Handwritten digits where the digits are unknown

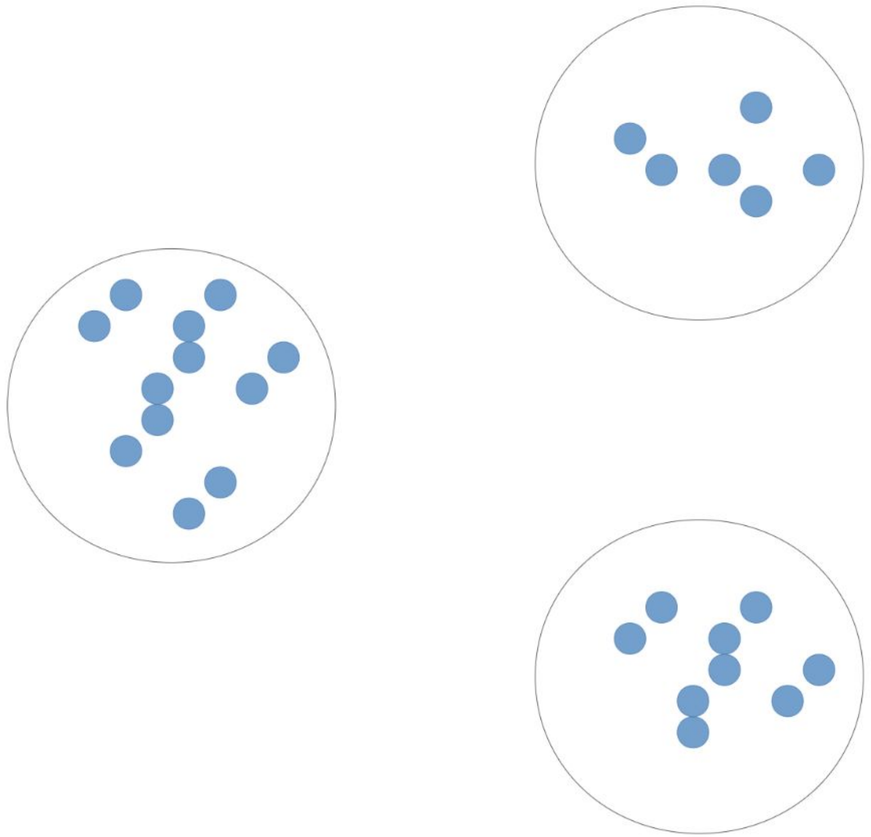


Question: What is the difference between Clustering and Classification?

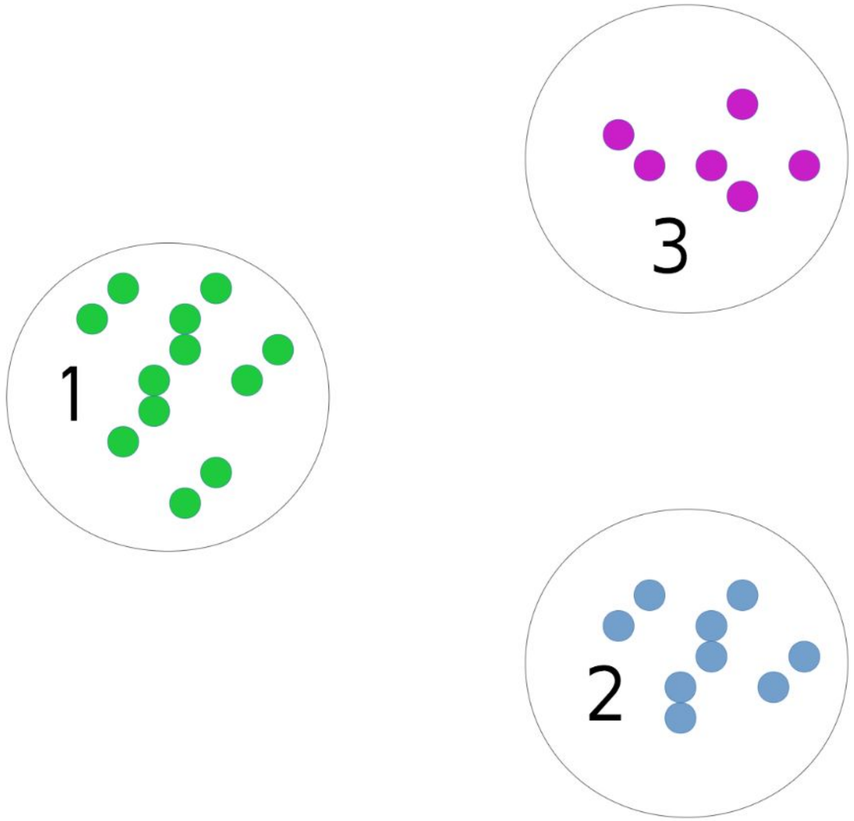
Clustering



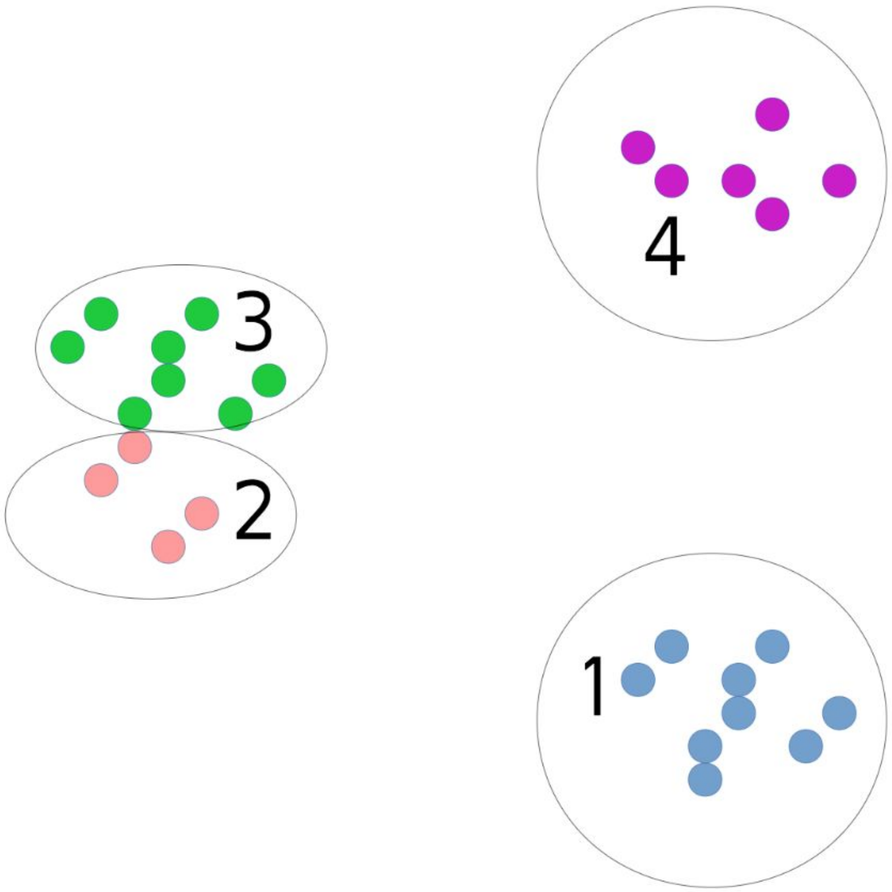
Clustering



Clustering



Clustering



Summary of Clustering

- Partition data into groups (clusters)
- Points within a cluster should be “similar”.
- Points in different cluster should be “different”.

Goal of Clustering

- Data Exploration
  - Are there coherent groups ?
  - How many groups are there ?
- Data Partitioning
  - Divide data by group before further processing

Outline

- Introduction to Clustering
- K-Means
  - K-Means Algorithm
  - Limitation of K-Means
  - K-Means Implementation
- Agglomerative Clustering
- Self-Organizing Maps (SOM)

Partitioning-based Clustering Problem

- Data: A collection of samples $x_i, for\ i = 1, \dots, n, where\ x_i \in R^d$
- Partition samples into K clusters, so that each cluster is as much cohesive as possible.

Question: How do we define the "cohesion" of a cluster?

Cohesion
- Cluster is cohesive

Similarity
- samples within cluster are similar to the cluster center

Closeness
- samples are close to corresponding cluster center

Distance
- distance between samples and cluster center are minimized

K-Means: Objective Function

- Assign each sample x_i to its closest cluster C_k with center μ_k , as to minimize the total within-cluster distance:

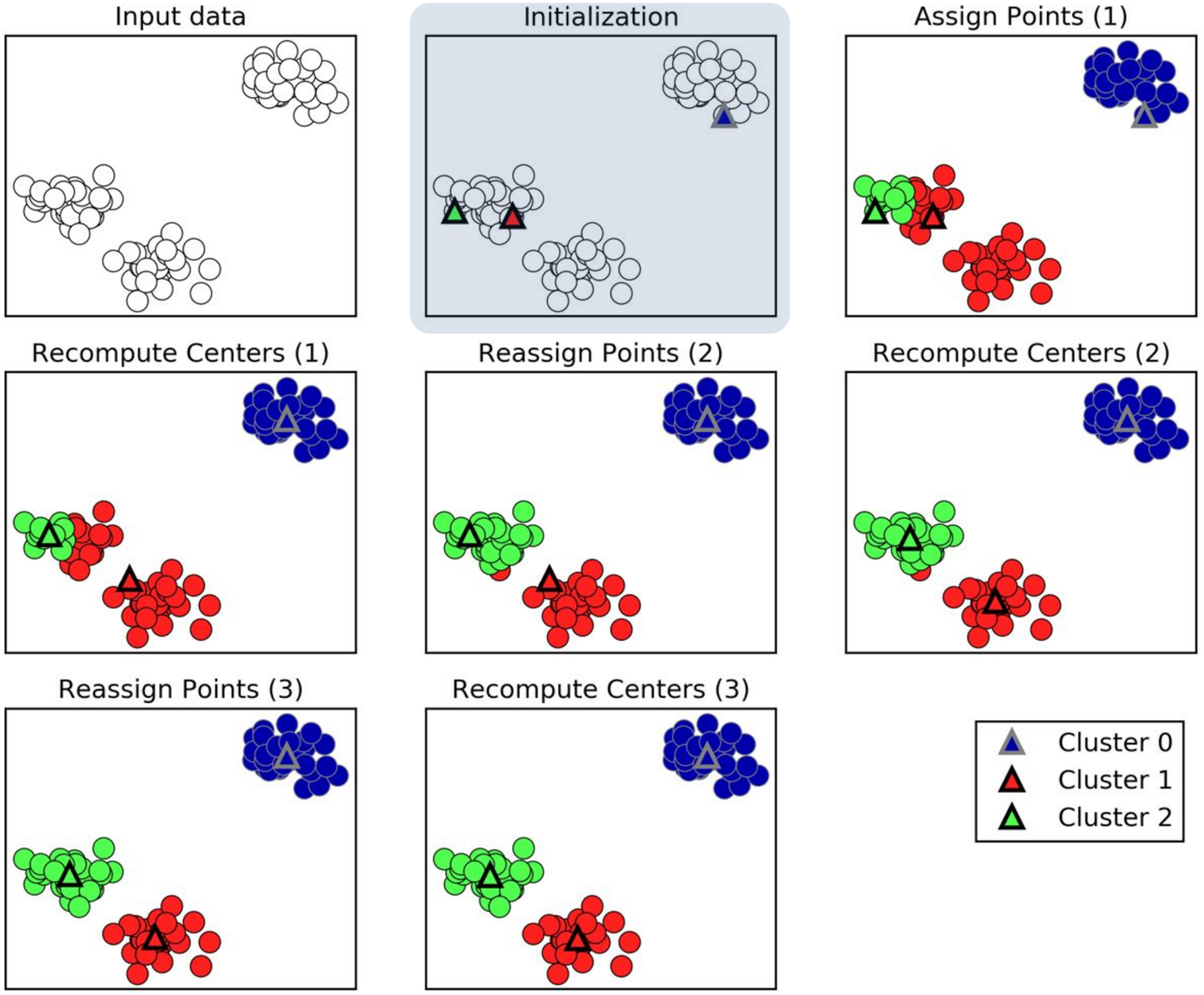
$$\arg\min_{\mu,C} \sum_{k=1}^{K} \sum_{x_i \in C_k} d(x_i, \mu_k)^2$$

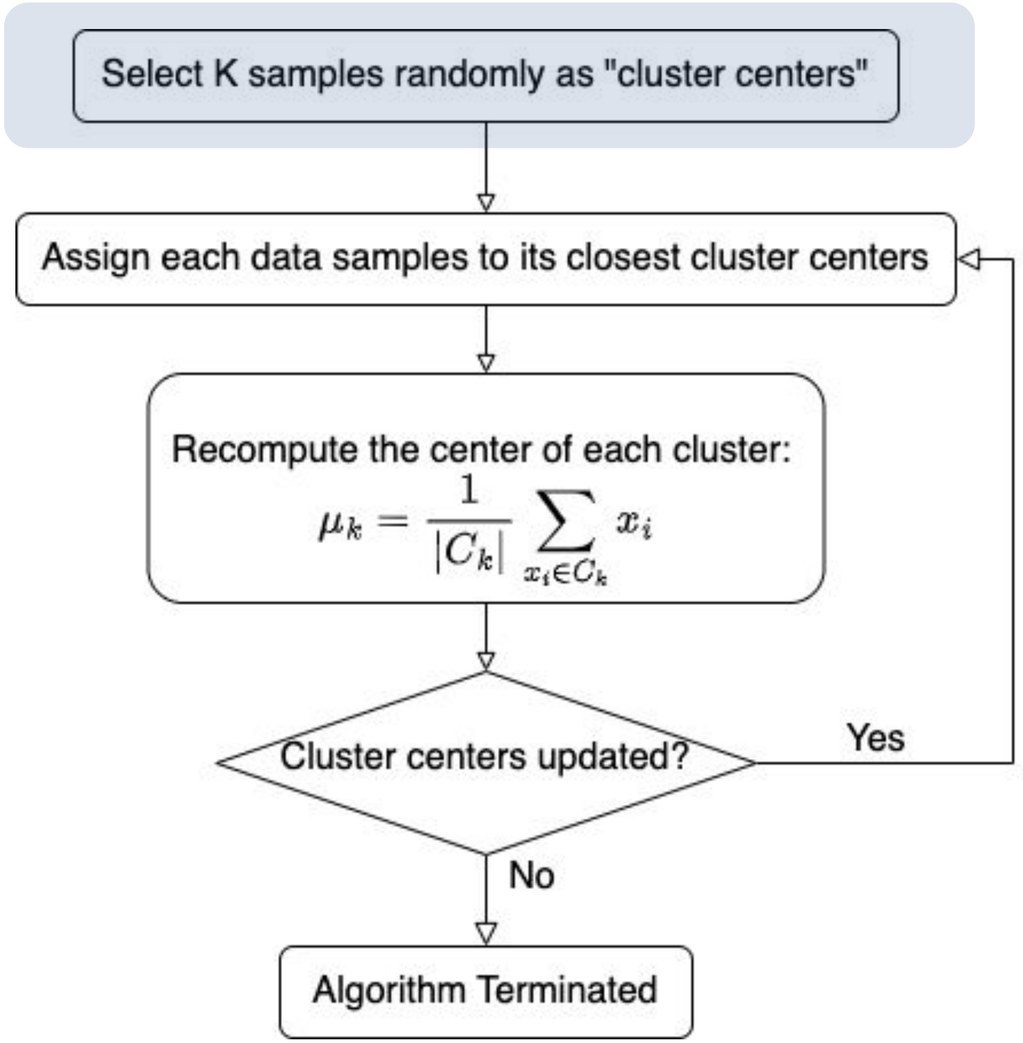
- Where d can be any distance/dissimilarity functions.
- In general, we suppose the data is on a Euclidean space. Then our objective is to minimize the following formula:

$$\arg\min_{\mu,C} \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

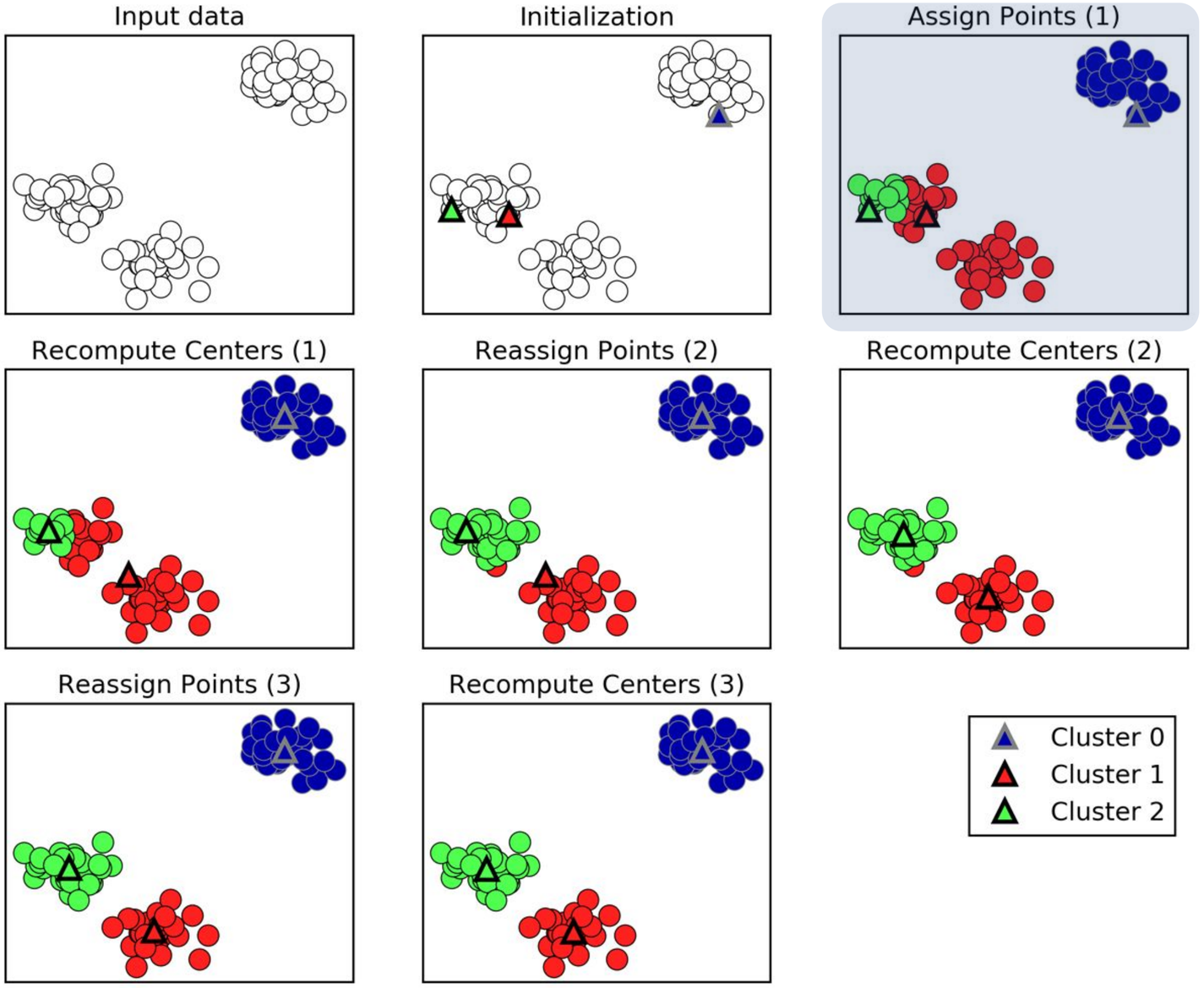
- We minimize the within-cluster Sum of Squared Error (SSE).
- For each point, the error is the distance to its nearest cluster center.

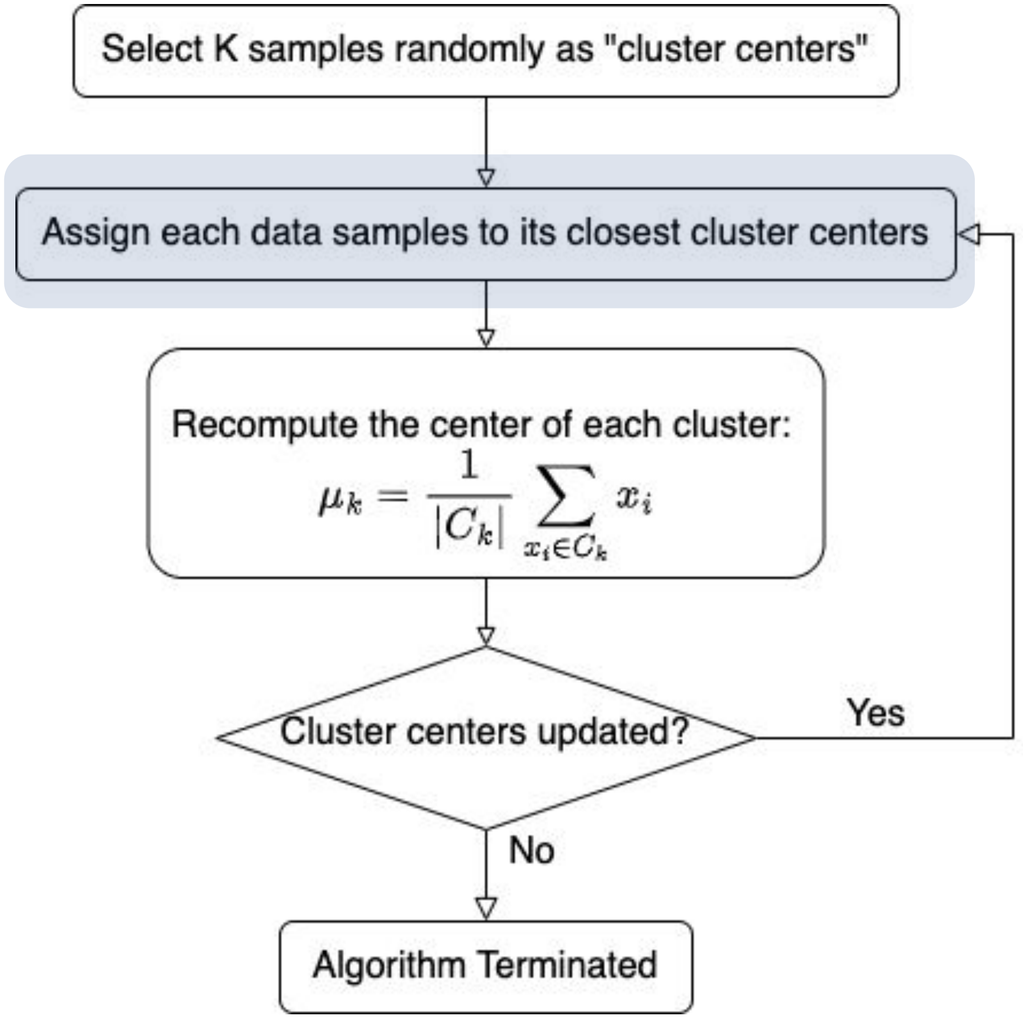
K-Means Algorithm



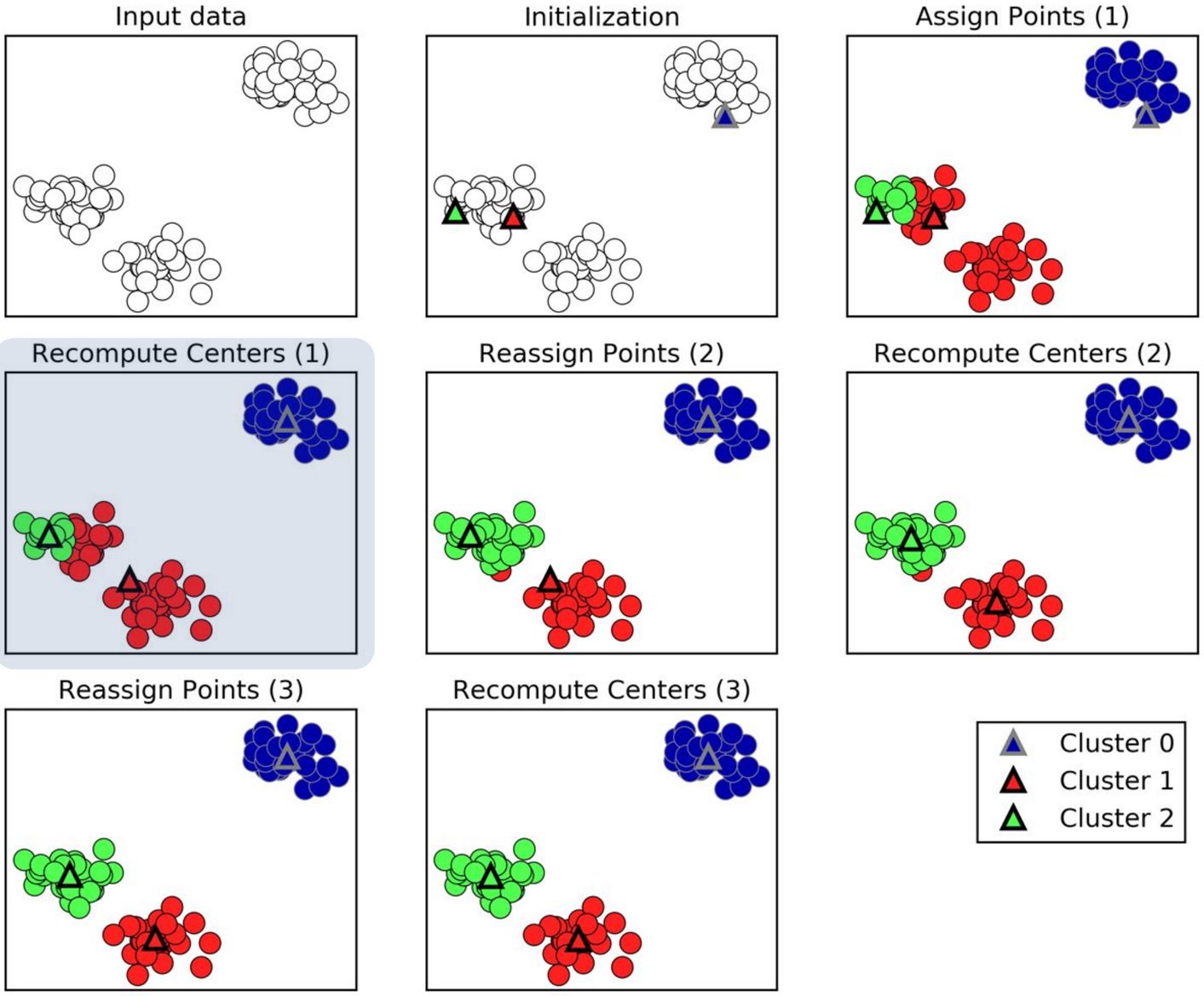


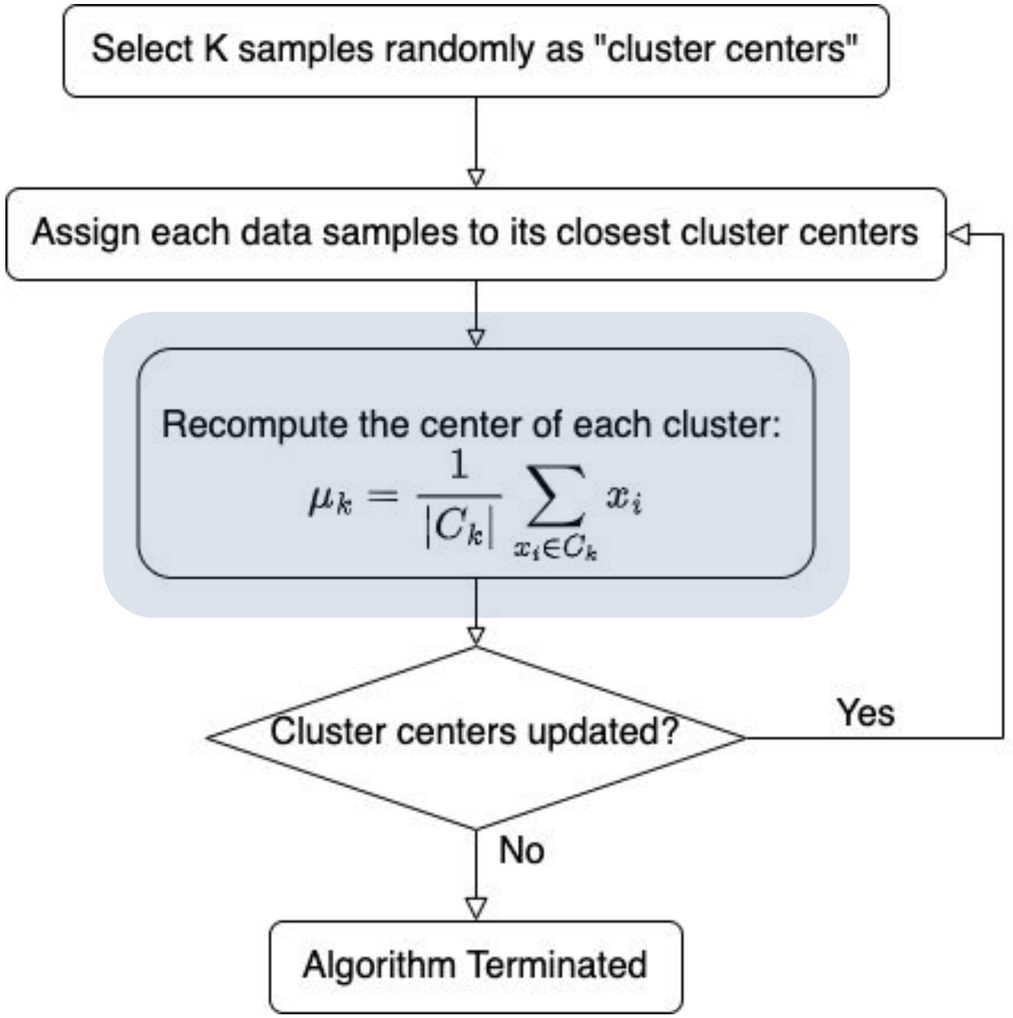
K-Means Algorithm



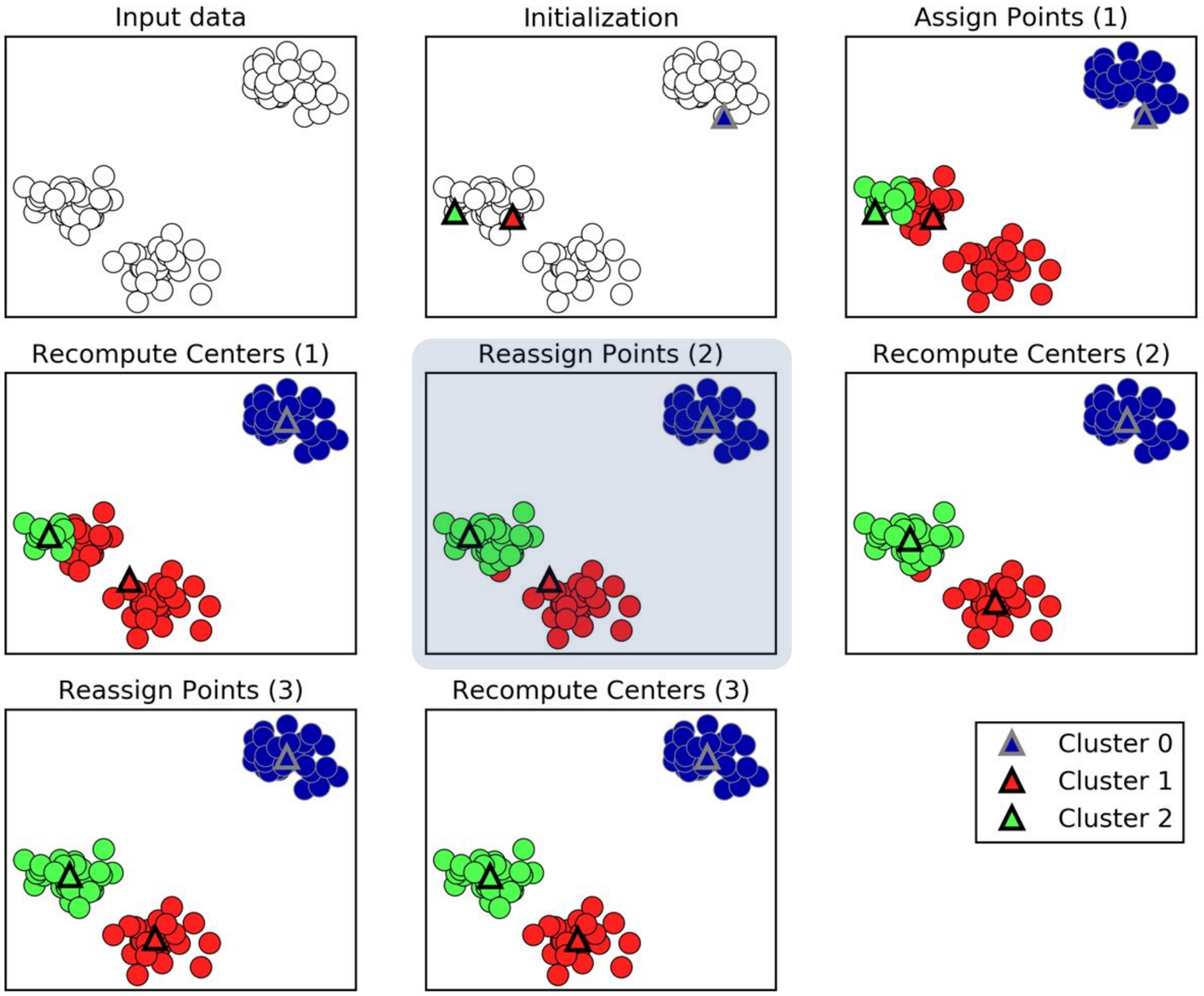


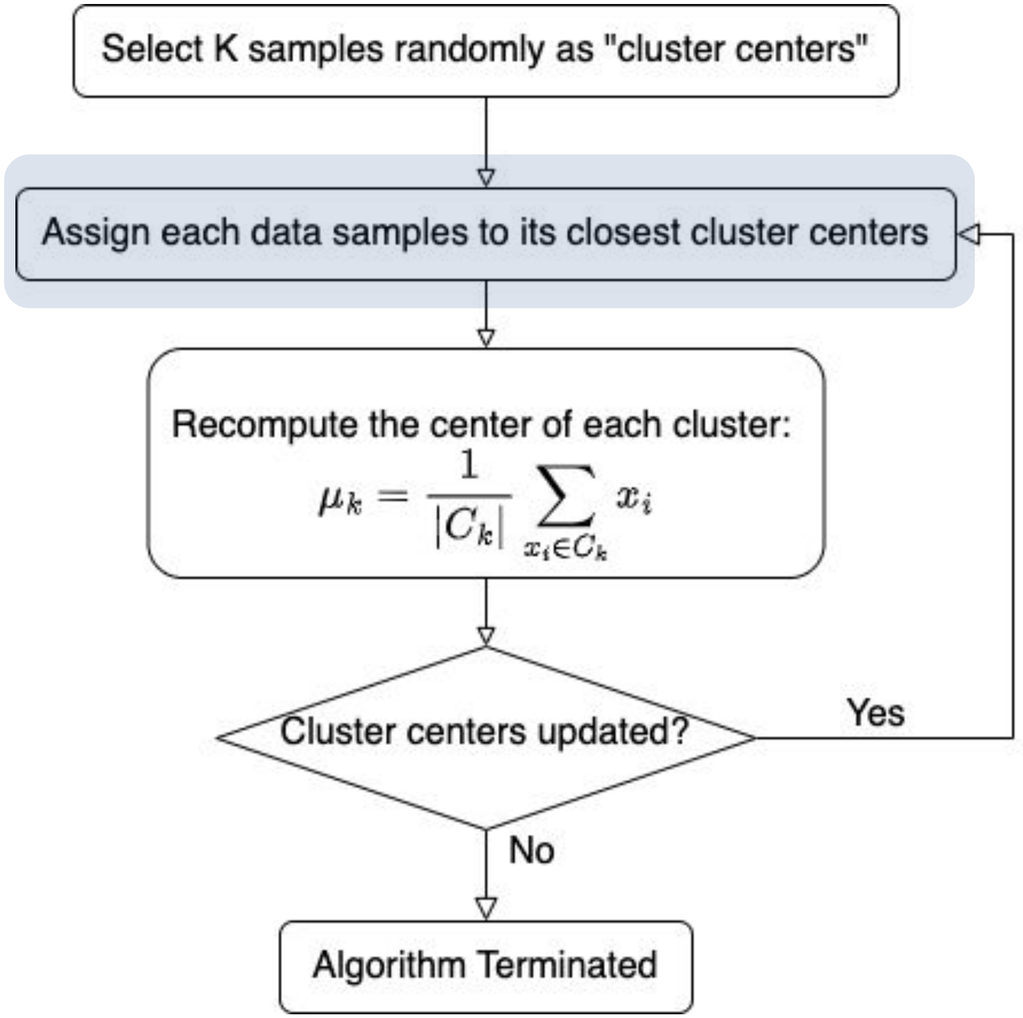
K-Means Algorithm



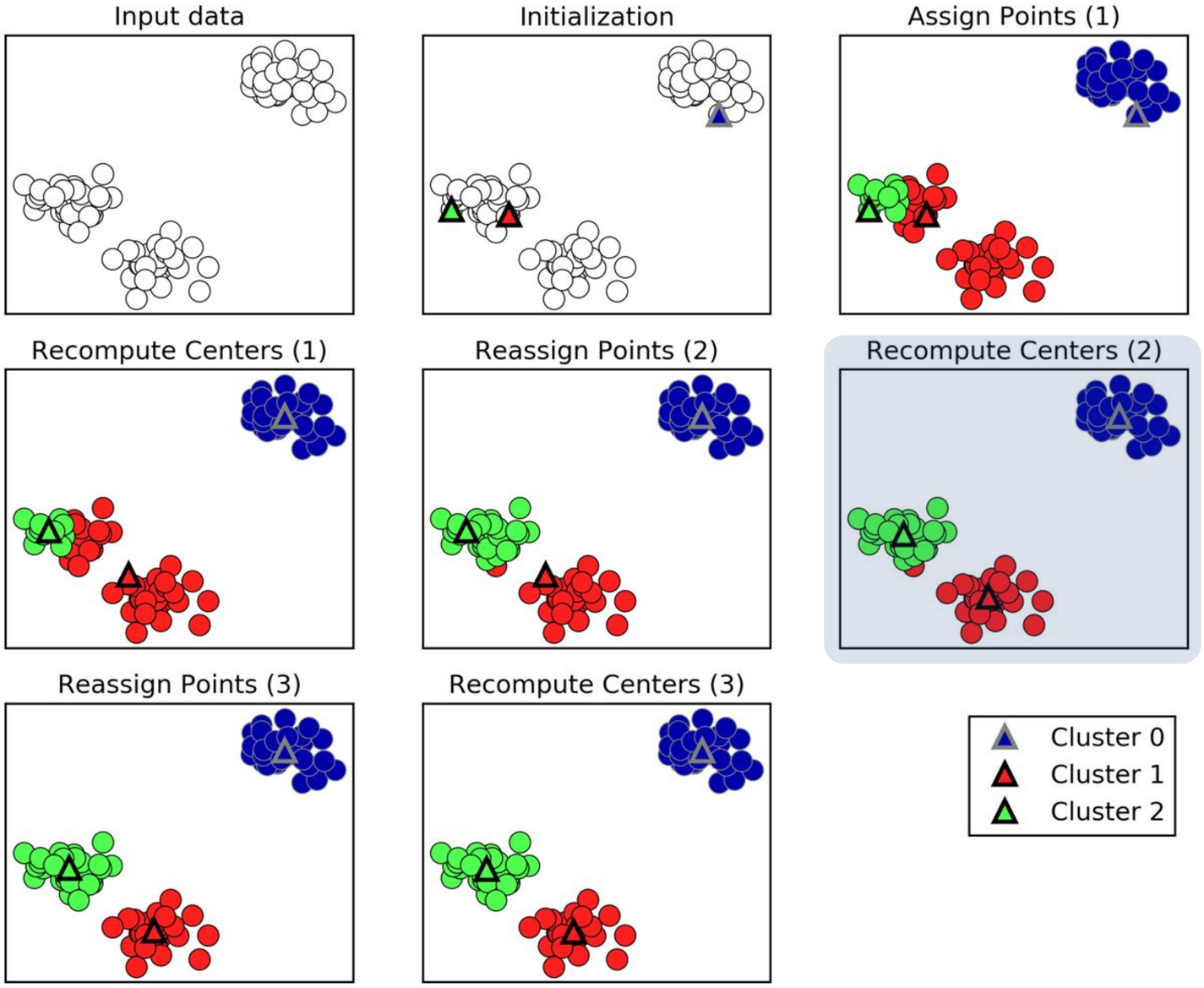


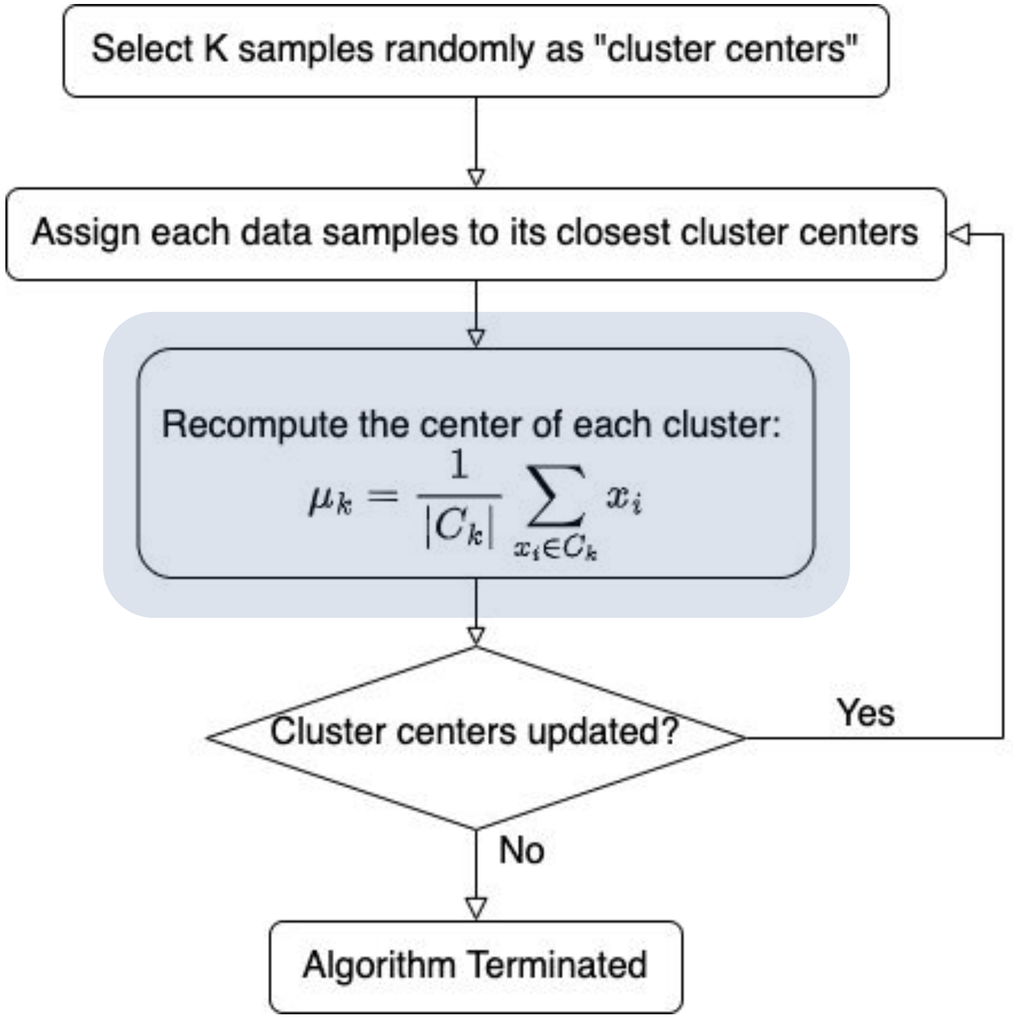
K-Means Algorithm



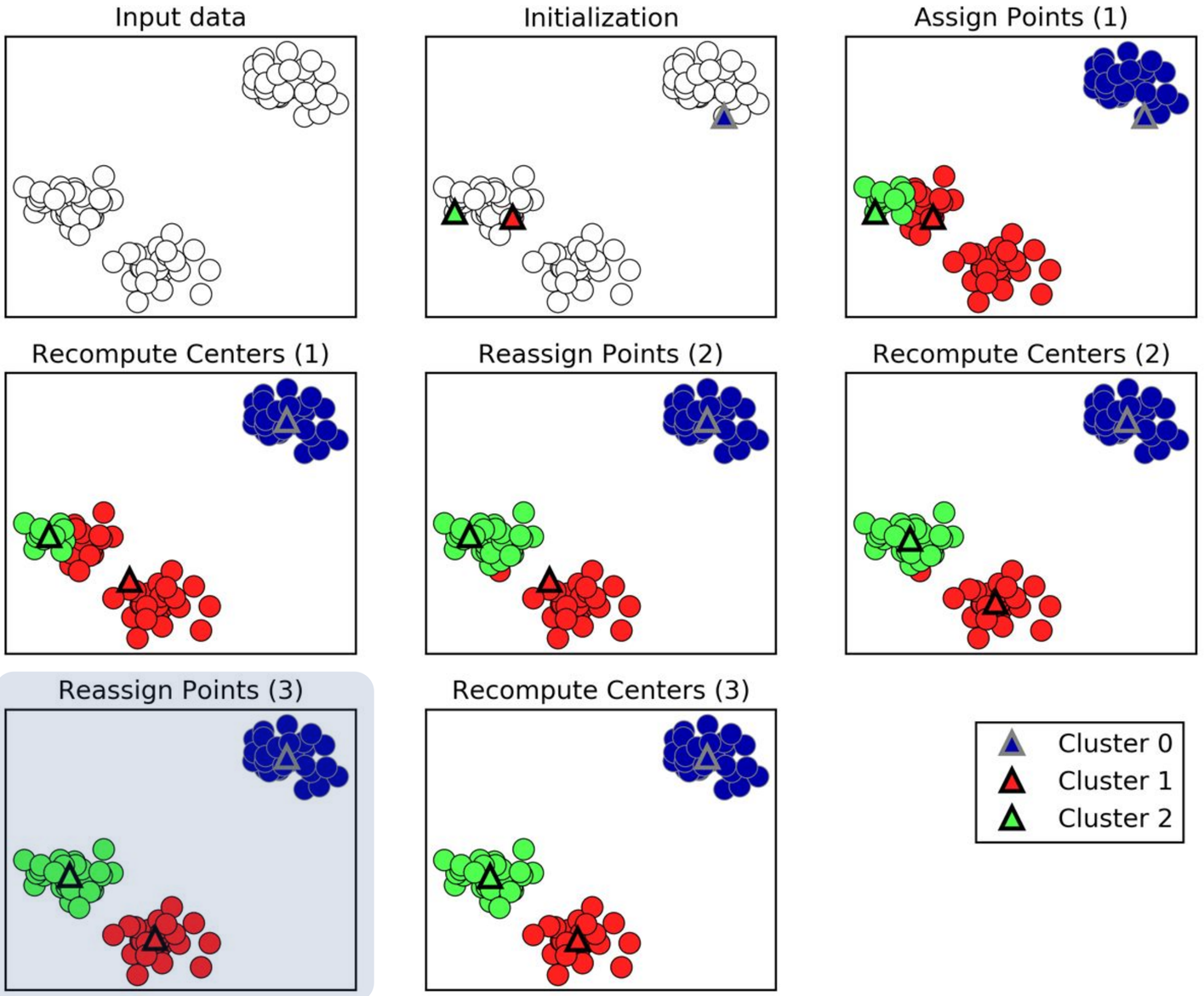


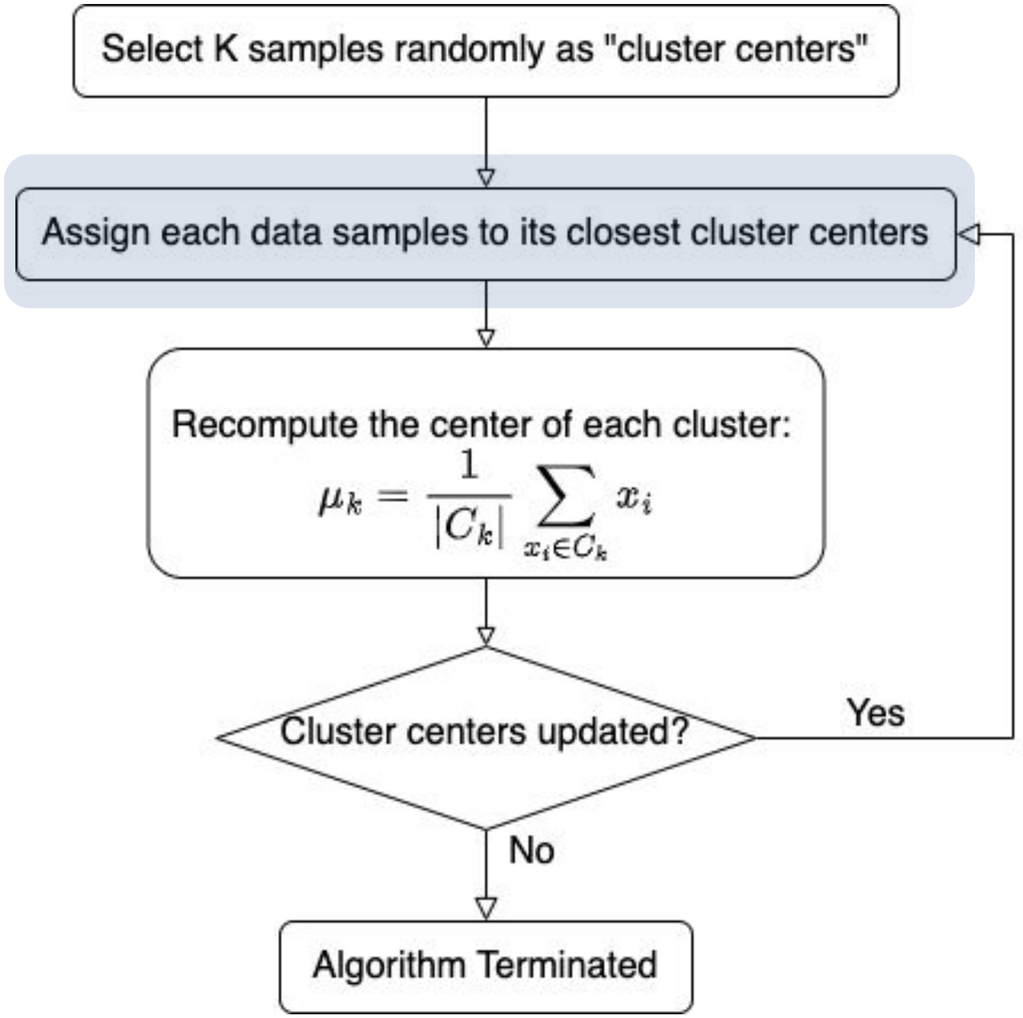
K-Means Algorithm



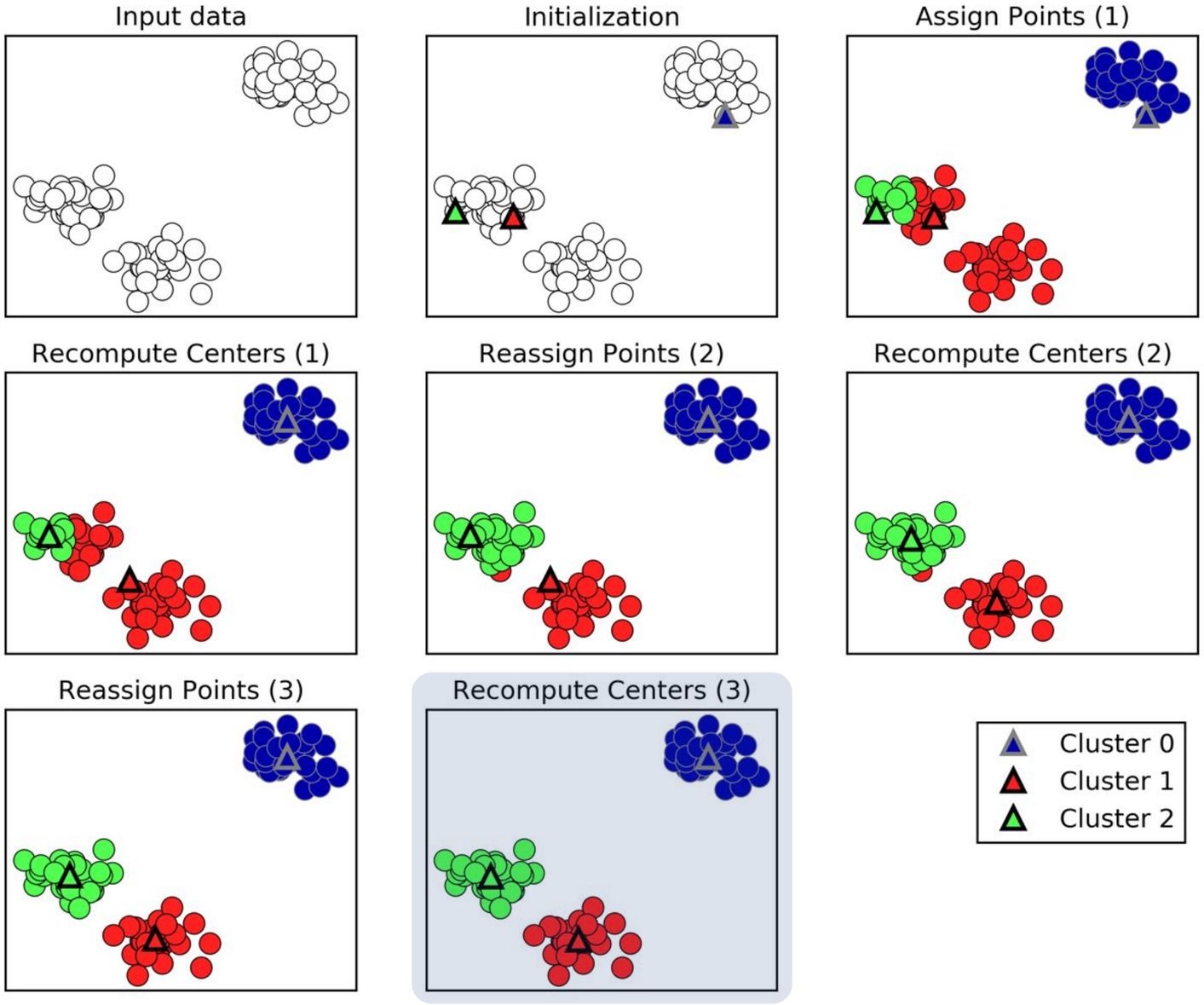


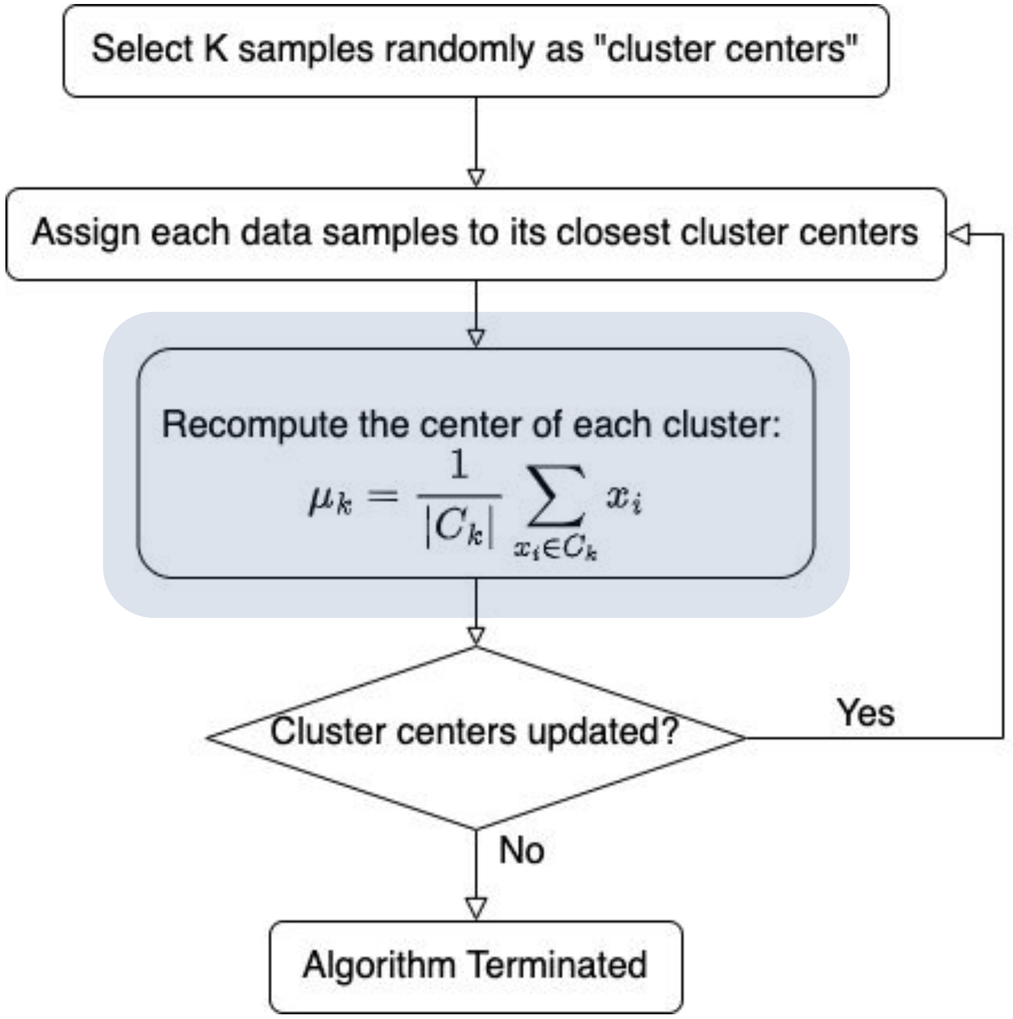
K-Means Algorithm





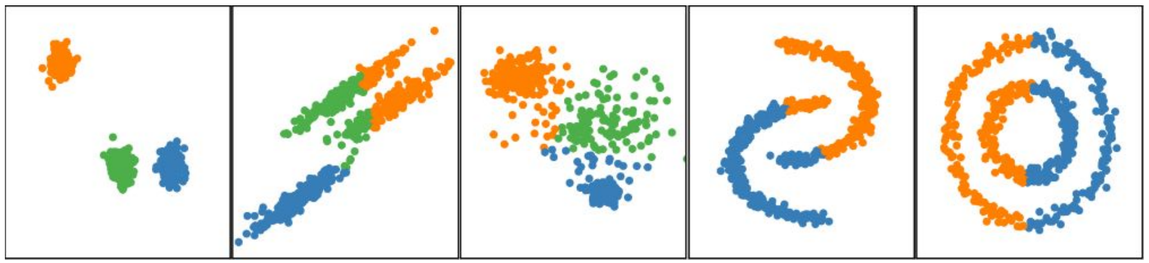
K-Means Algorithm



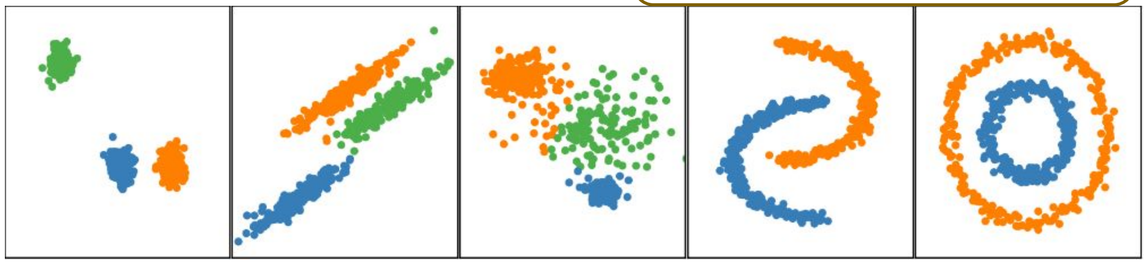


K-Means: Strength and Limitations

k-means Clustering $(t_{avg} = 0.028s)$



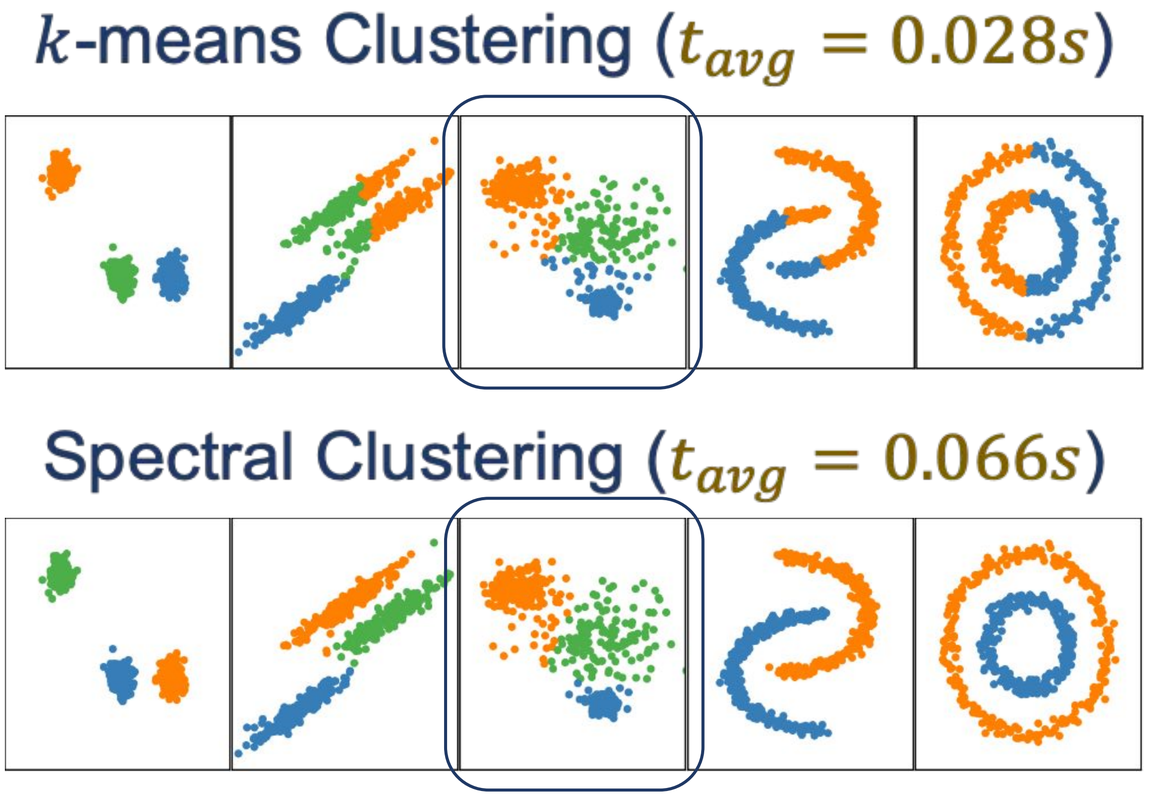
Spectral Clustering $(t_{avg} = 0.066s)$



Strengths:

1. Simple and Efficient.

K-Means: Strength and Limitations



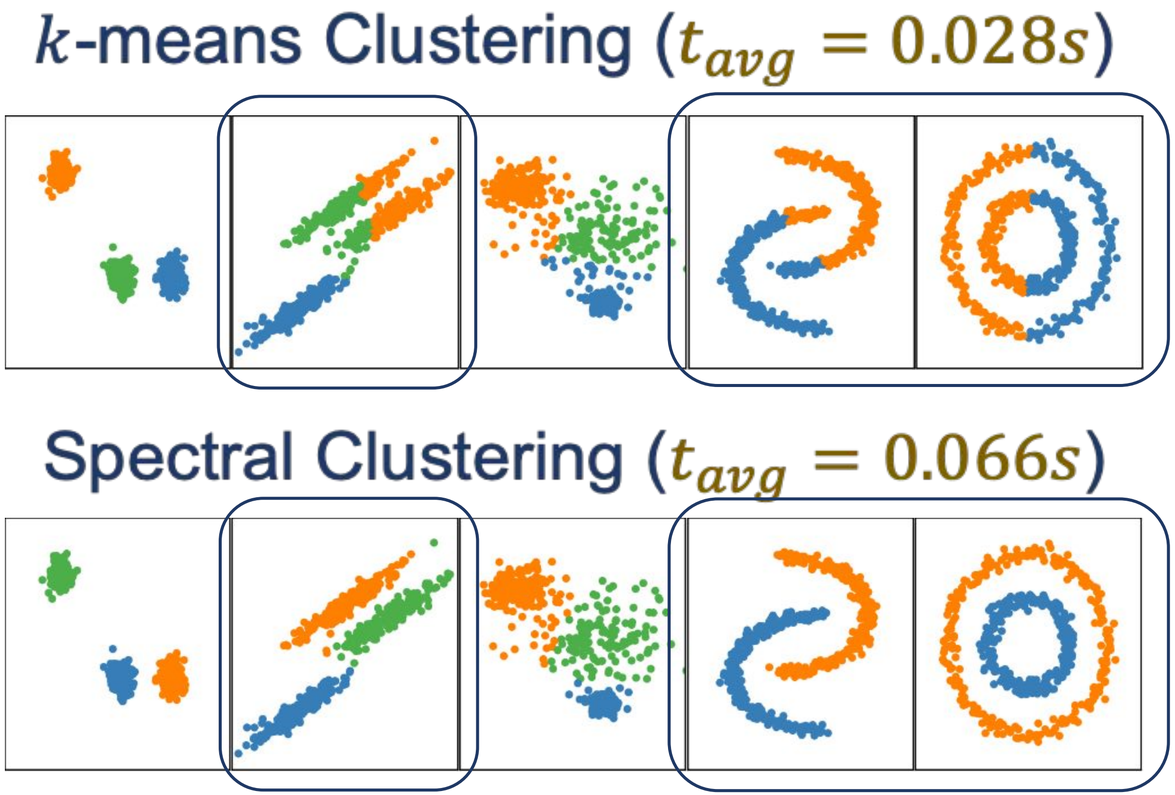
Strengths:

1. Simple and Efficient.

Weaknesses:

1. Clusters with different sizes and densities.

K-Means: Strength and Limitations



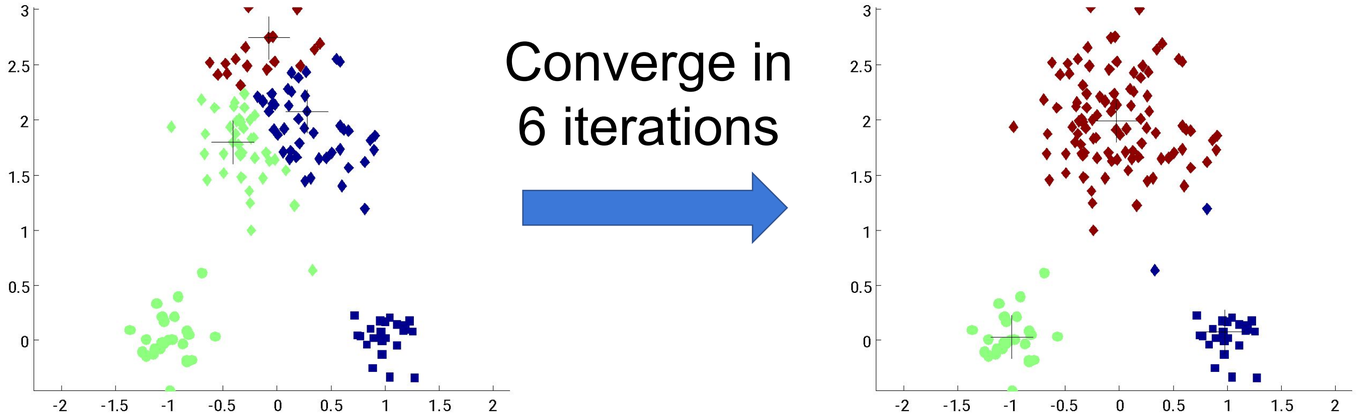
Strengths:

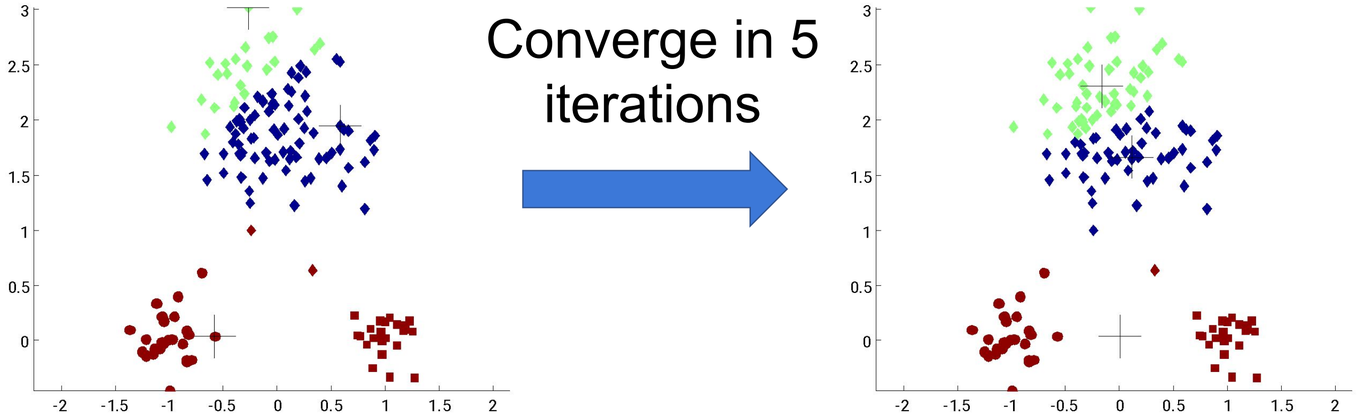
1. Simple and Efficient.

Weaknesses:

1. Clusters with different sizes and densities.
2. Non-spherical clusters.

K-Means: Strength and Limitations





Strengths:

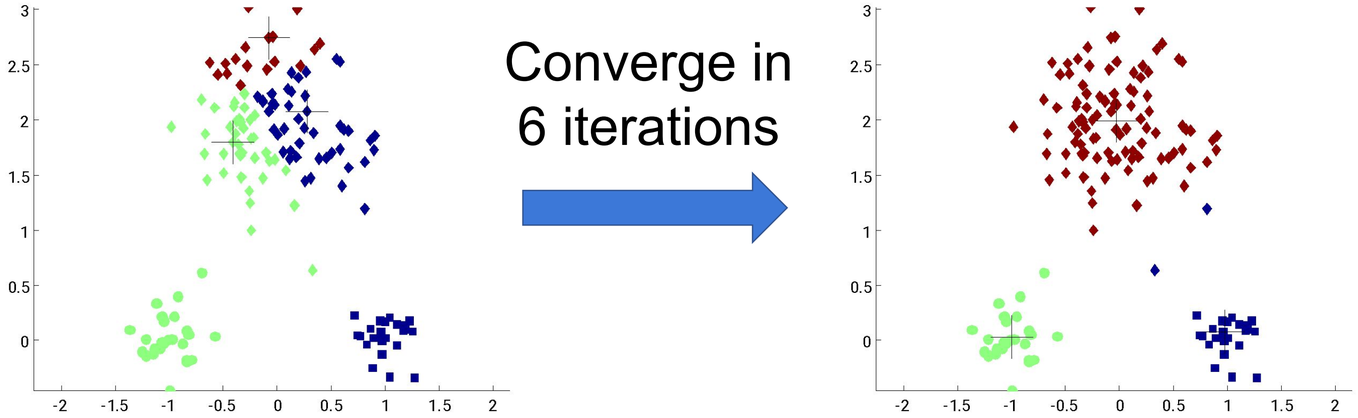
1. Simple and Efficient.

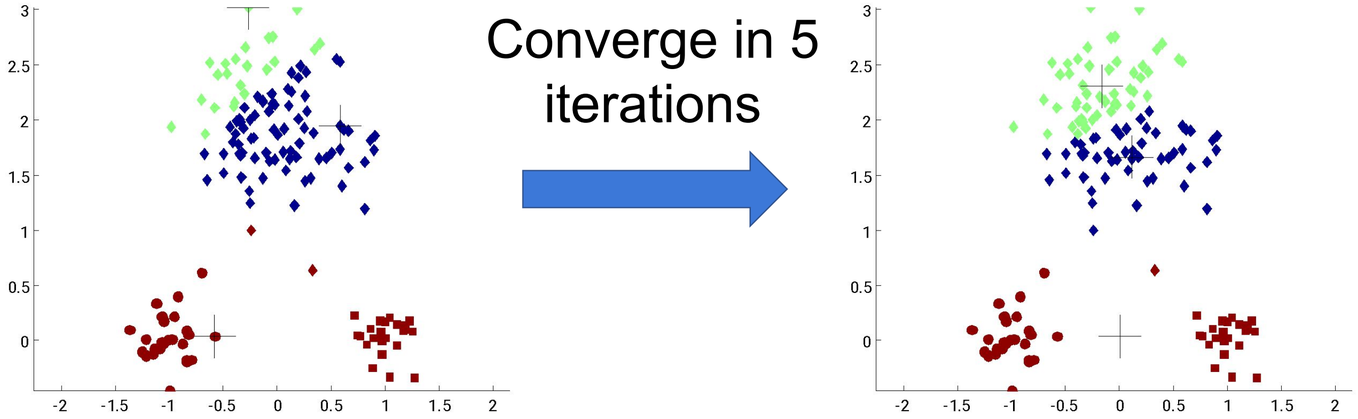
Weaknesses:

1. Clusters with different sizes and densities.
2. Non-spherical clusters.
3. Sensitive to initial centroids.

Question: How can we deal with the initialization problem?

K-Means: Strength and Limitations

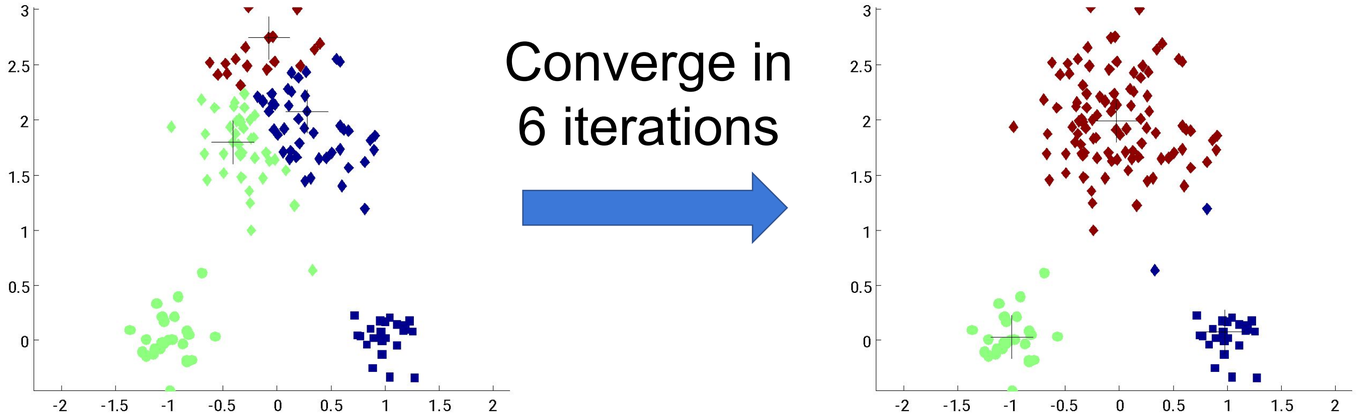


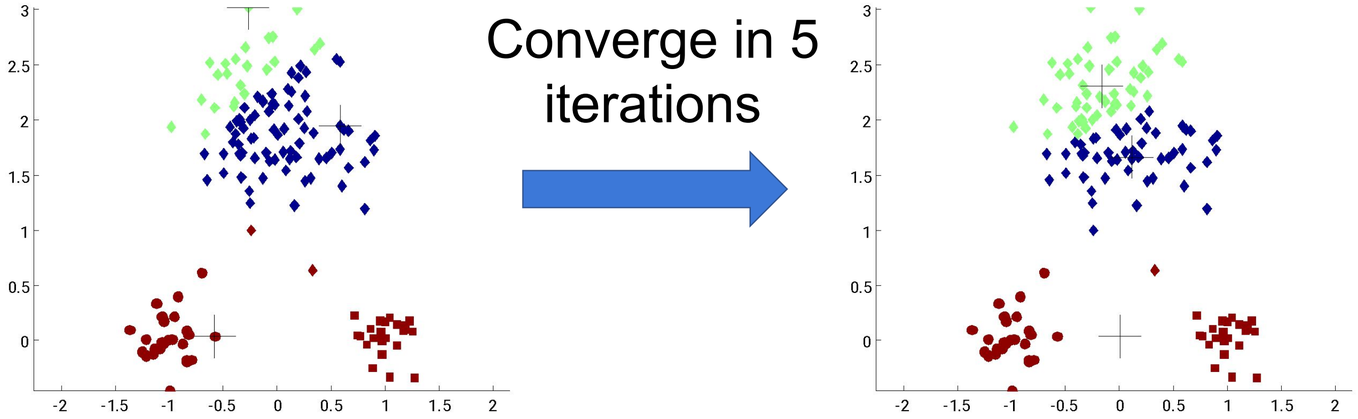


Question: How can we deal with the initialization problem?

1. Multi-start with best result.

K-Means: Strength and Limitations

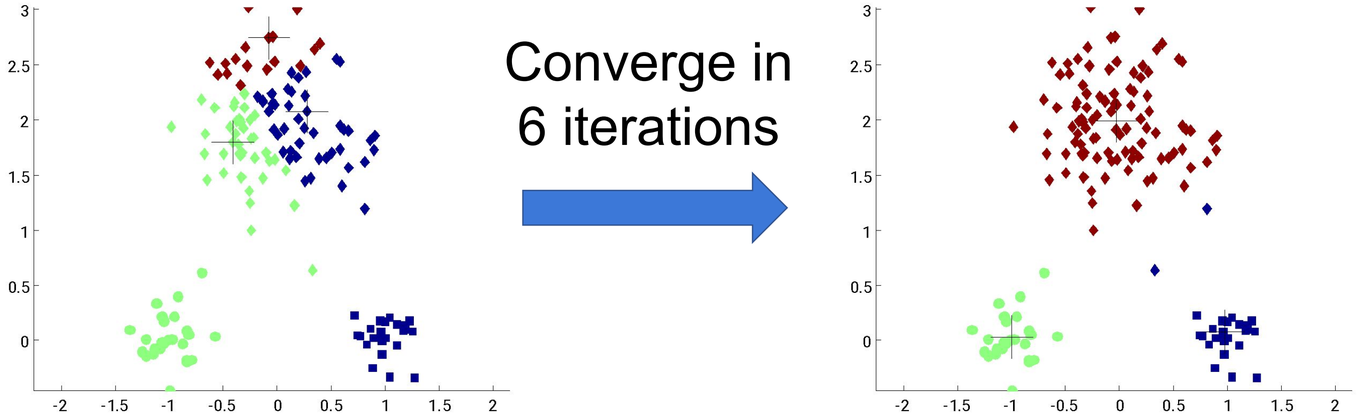


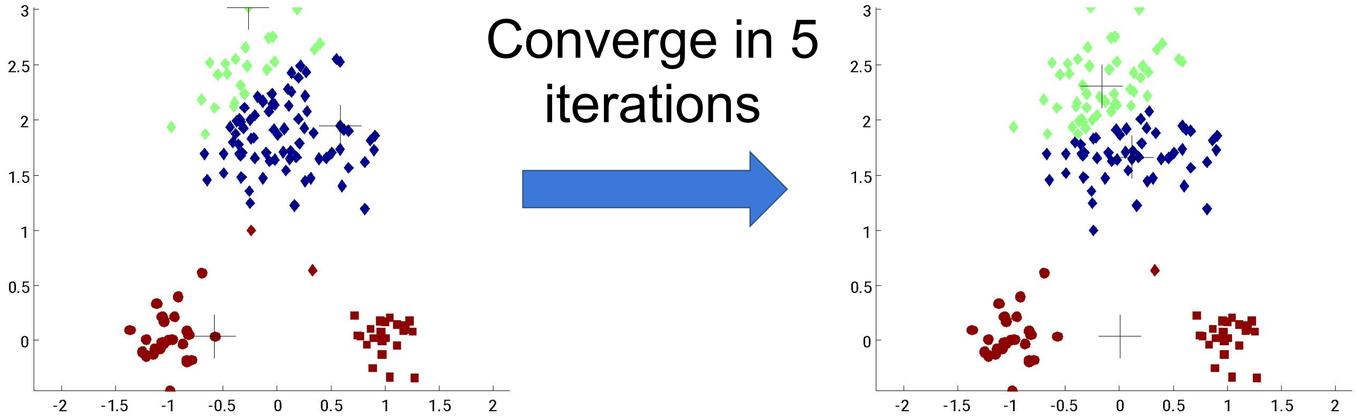


Question: How can we deal with the initialization problem?

1. Multi-start with best result.
2. Heuristic for initial centers selection: K-Means++

K-Means: Strength and Limitations

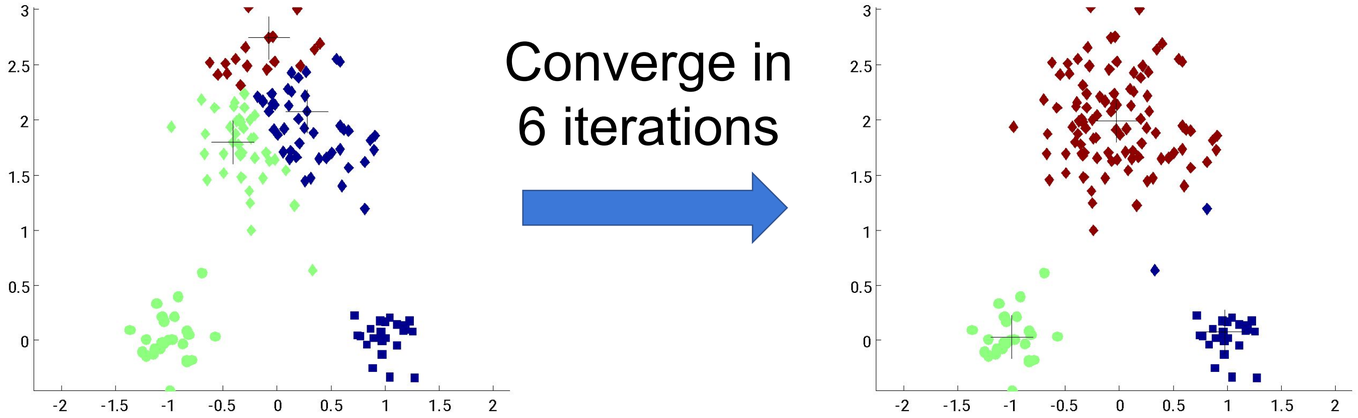


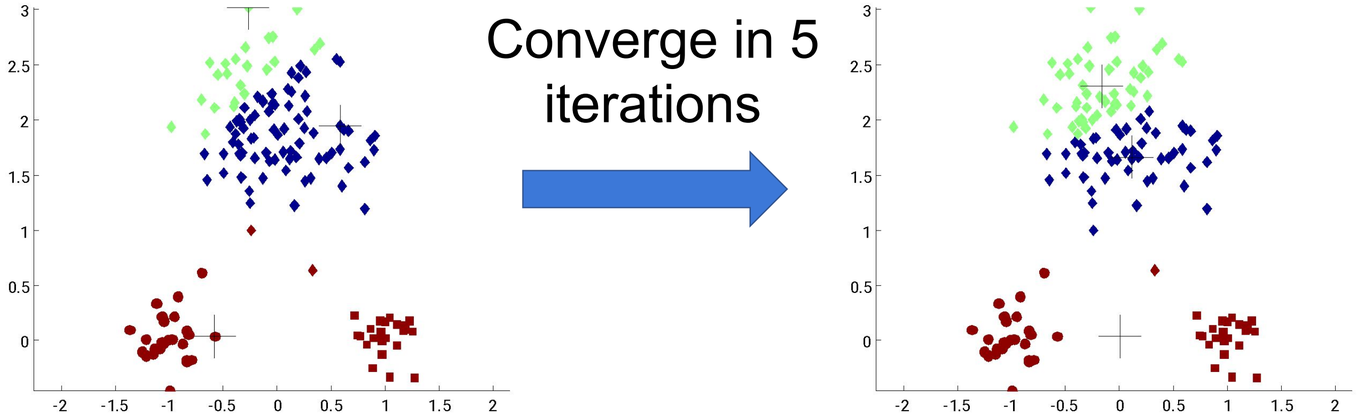


Question: How can we deal with the initialization problem?

1. Multi-start with best result.
2. Heuristic for initial centers selection: K-Means++
3. Algorithm invariant to initial selection: Bisecting K-Means

K-Means: Strength and Limitations

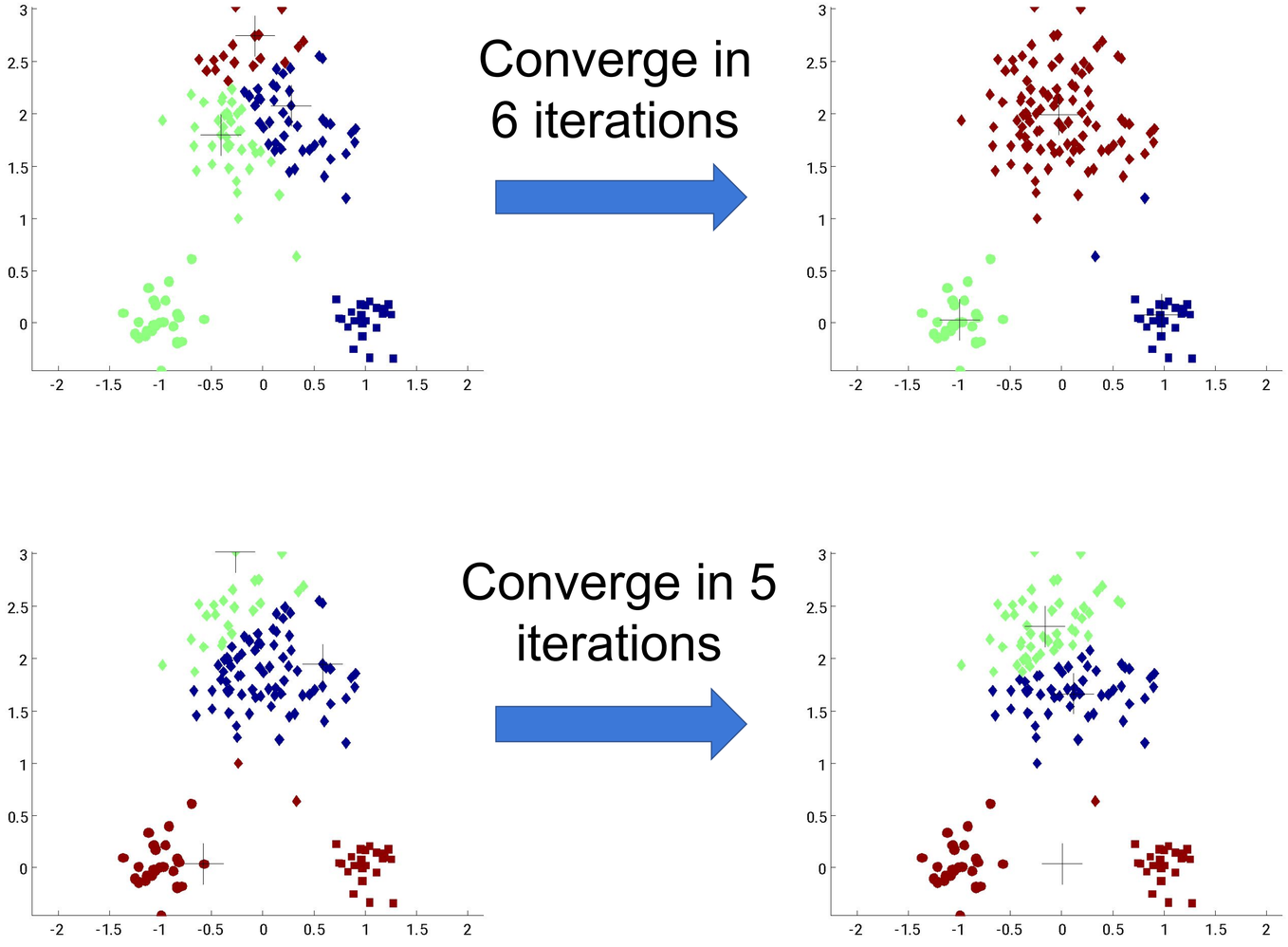




Question: How can we deal with the initialization problem?

1. Multi-start with best result.
2. Heuristic for initial centers selection: K-Means++
3. Algorithm invariant to initial selection: Bisecting K-Means
4. Post processing on clusters

K-Means: Strength and Limitations



Question: How can we deal with the initialization problem?

1. Multi-start with best result.
2. Heuristic for initial centers selection: K-Means++
3. Algorithm invariant to initial selection: Bisecting K-Means
4. Post processing on clusters
5. Global optimization

K-Means API

```
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
```

```
X, y = make_blobs(centers=4, random_state=1)
```

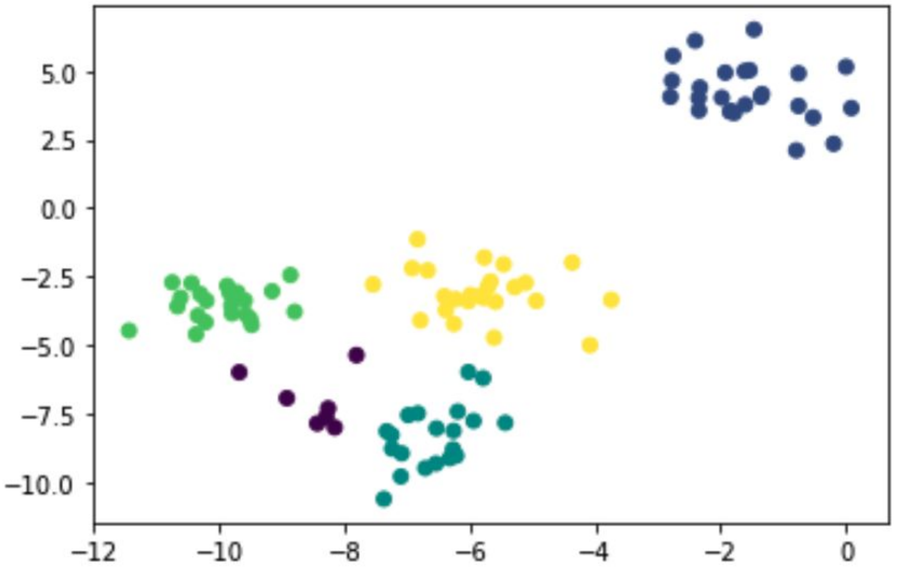
```
km = KMeans(n_clusters=5, random_state=0)
km.fit(X)
print(km.cluster_centers_.shape)
print(km.labels_.shape)
```

```
(5, 2)
(100,)
```

```
# predict is the same as labels_ on training data
# but can be applied to new data
print(km.predict(X).shape)
```

```
(100,)
```

```
plt.scatter(X[:, 0], X[:, 1], c=km.labels_)
```



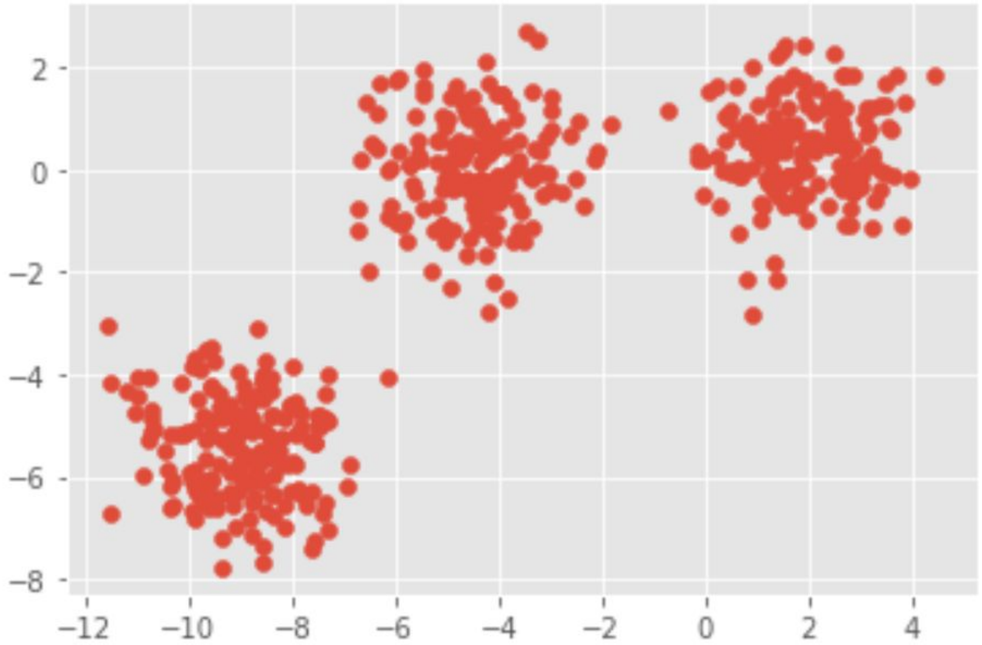
Initialization

- Random centers fast
- K-means++ (default): Greedily add ‘furthest way’ point
- K-means++ initialization may take much longer than clustering

K-Means Application

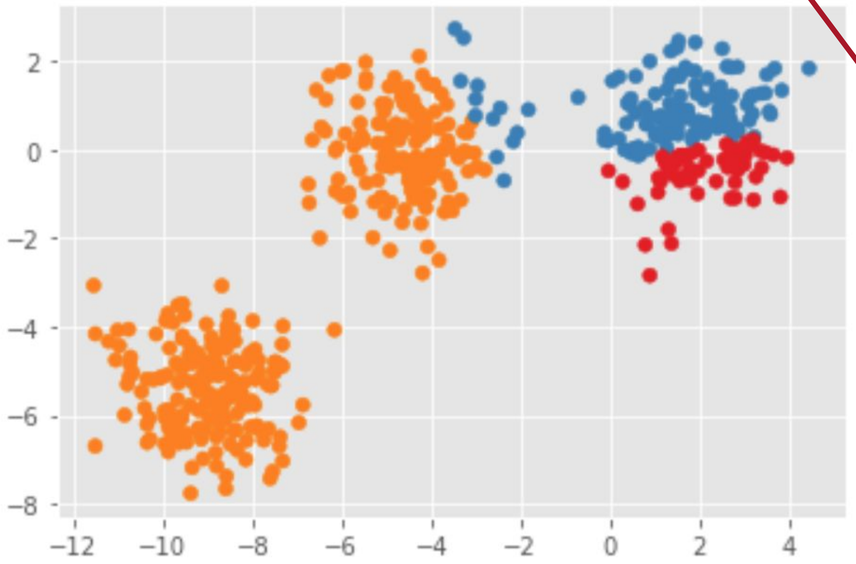
```
# make_blobs generates gaussian blobs, we create 3 blobs
n_samples = 500
random_state = 170
X, y = make_blobs(n_samples=n_samples, centers=3, random_state=random_state)

# plot data
plt.scatter(X[:, 0], X[:, 1],  marker="o");
```



K-Means Application

```
y_pred = KMeans(n_clusters=3, n_init=1, init='random', max_iter=1).fit_predict(X)
plt.scatter(X[:, 0], X[:, 1], c=y_pred,  marker="o"  cmap=cmap);
```

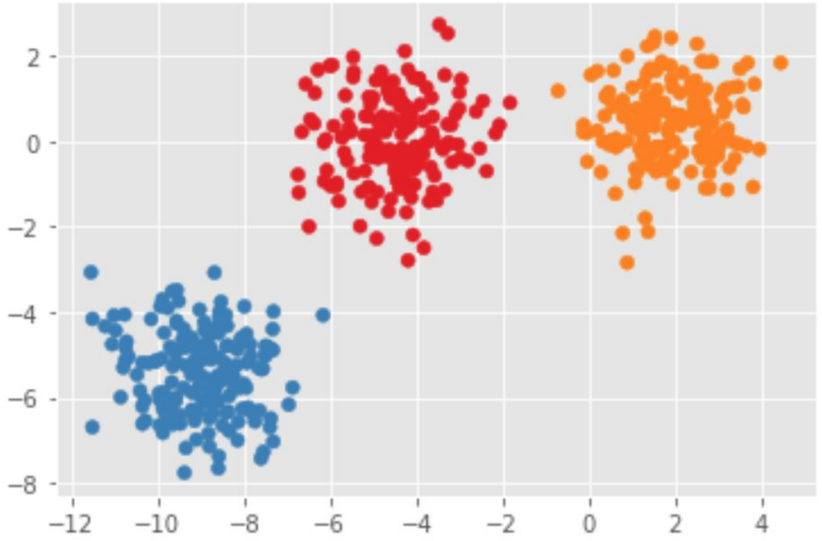


- random
- k-means++ by default

- Number of iterations in each initialization

- Number of initialization
- 10 by default

```
y_pred = KMeans(n_clusters=3, n_init=1, init='random', max_iter=5).fit_predict(X)
plt.scatter(X[:, 0], X[:, 1], c=y_pred,  marker="o", cmap=cmap);
```



Outline

- Introduction to Clustering
- K-Means
  - K-Means Algorithm
  - Limitation of K-Means
  - K-Means Implementation
- Agglomerative Clustering
- Self-Organizing Maps (SOM)

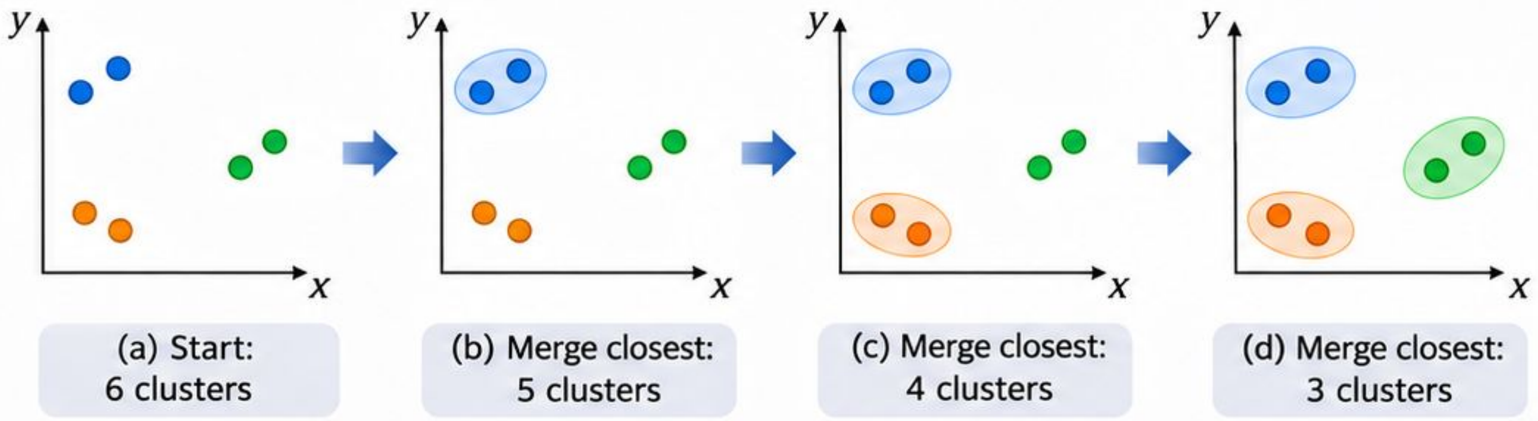
Agglomerative Clustering: Overview

- **Key idea**
  - A hierarchical (**bottom-up**) clustering **method**
  - Start with each data point as a cluster, and iteratively merge the two closest clusters until the desired number of clusters is reached

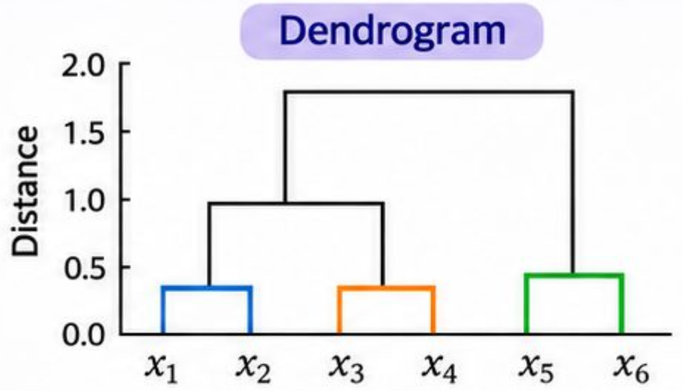
Algorithm

1. **Initialize:** each data point is a cluster.
2. **Compute** the distance between all clusters (e.g., using single, complete, or average linkage).
3. **Merge** the two closest clusters.
4. **Repeat** Steps 2–3 until the stopping condition is met (e.g., a target number of clusters K).

Example: Merging Process (Single Linkage)

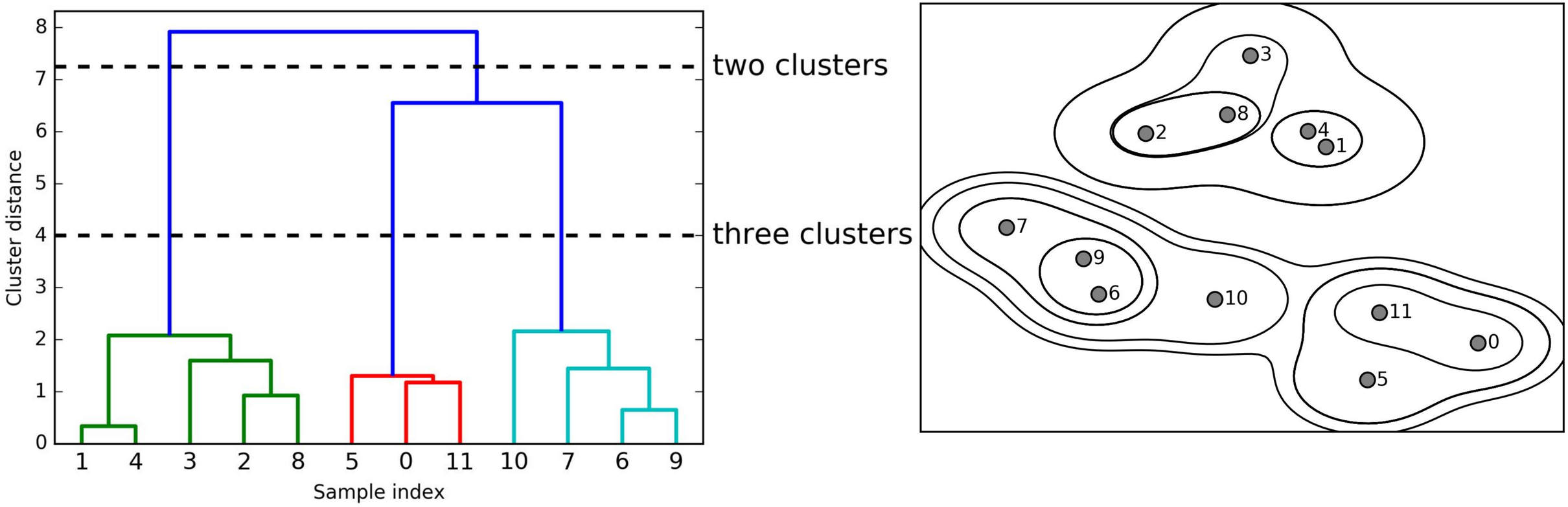


(a) Start: 6 clusters | (b) Merge closest: 5 clusters | (c) Merge closest: 4 clusters | (d) Merge closest: 3 clusters

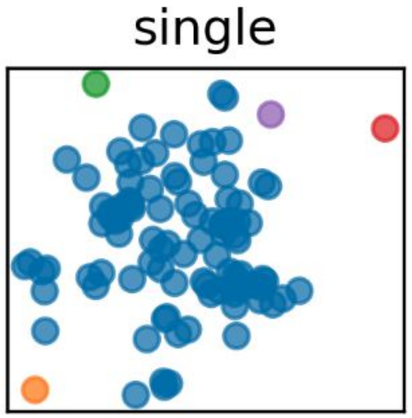


- **Output:** a hierarchy of clusters (dendrogram), from which we can choose a number of clusters K.

Dendrograms



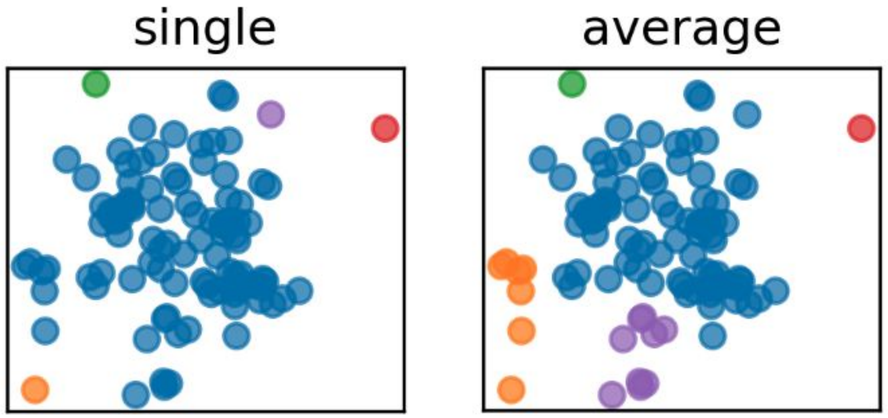
Merging (Linkage) Criteria




```
single :   [96  1  1  1  1]
```

- Single Linkage
  - Smallest minimum distance

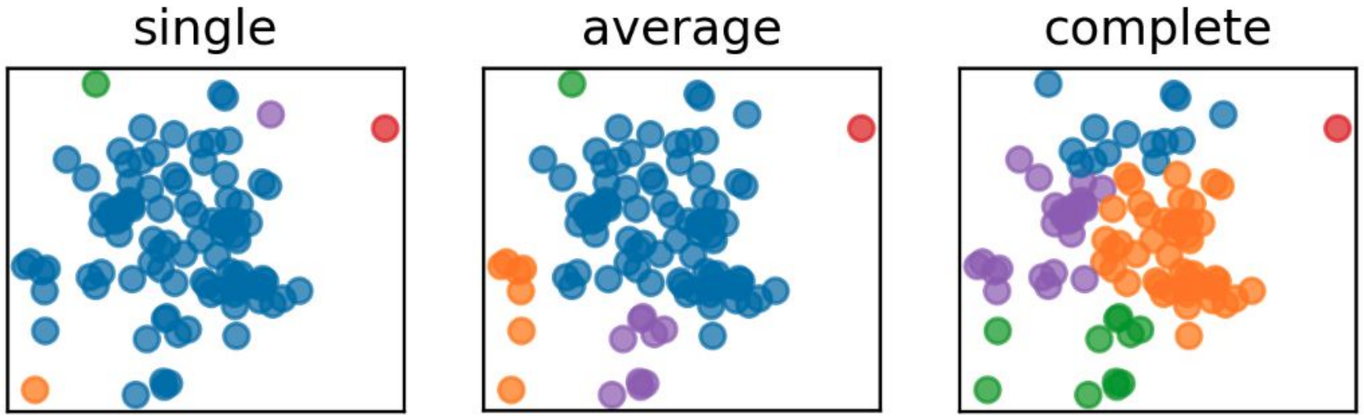
Merging (Linkage) Criteria




```
single :   [96  1  1  1  1]
average :  [82  9  7  1  1]
```

- Single Linkage
  - Smallest minimum distance
- Average Linkage
  - Smallest average distance between all pairs in the clusters

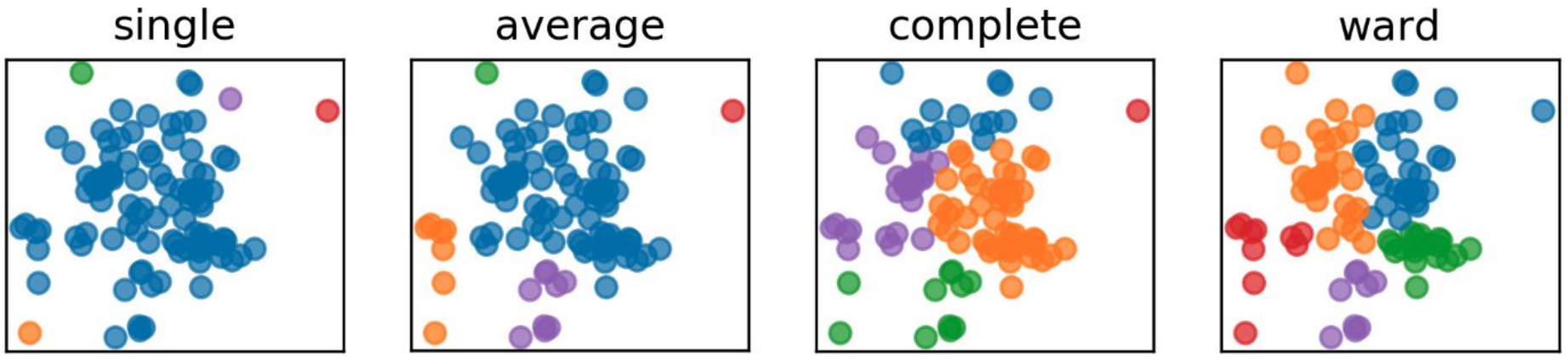
Merging (Linkage) Criteria




```
single :   [96  1  1  1  1]
average :  [82  9  7  1  1]
complete : [50 24 14 11  1]
```

- Single Linkage
  - Smallest minimum distance
- Average Linkage
  - Smallest average distance between all pairs in the clusters
- Complete Linkage
  - Smallest maximum distance

Merging (Linkage) Criteria




```
single :   [96  1  1  1  1]
average :  [82  9  7  1  1]
complete : [50 24 14 11  1]
ward :     [31 30 20 10  9]
```

- Single Linkage
  - Smallest minimum distance
- Average Linkage
  - Smallest average distance between all pairs in the clusters
- Complete Linkage
  - Smallest maximum distance
- Ward (default in sklearn)
  - Smallest increase in within-cluster variance
  - Leads to more equally sized clusters.

Agglomerative Clustering: Summary

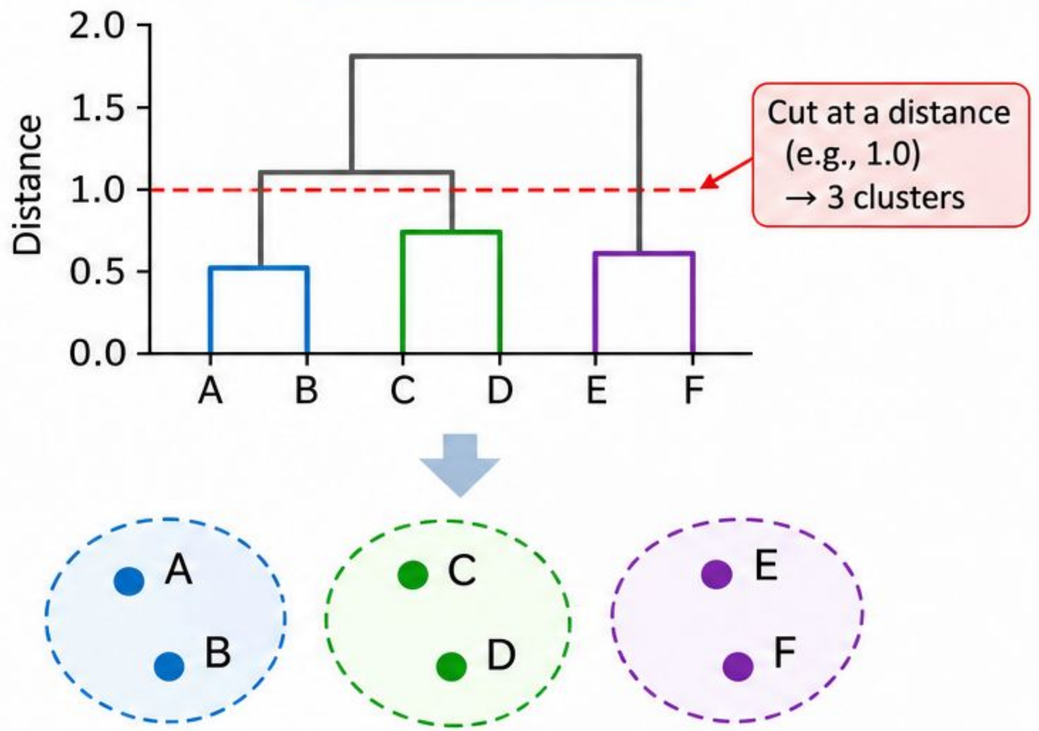
Key Idea

- A bottom-up (hierarchical) method that iteratively merges the most similar clusters until one cluster remains.

Algorithm (Overview)

1. Start with each sample as a cluster.
2. Compute pairwise cluster distances (e.g., single, complete, average).
3. Merge the two closest clusters.
4. Repeat until all samples are merged.

Dendrogram



Advantages

- Does not require specifying the number of clusters.
- Provides a hierarchy of clusters (dendrogram), allowing different levels of granularity.

Limitations

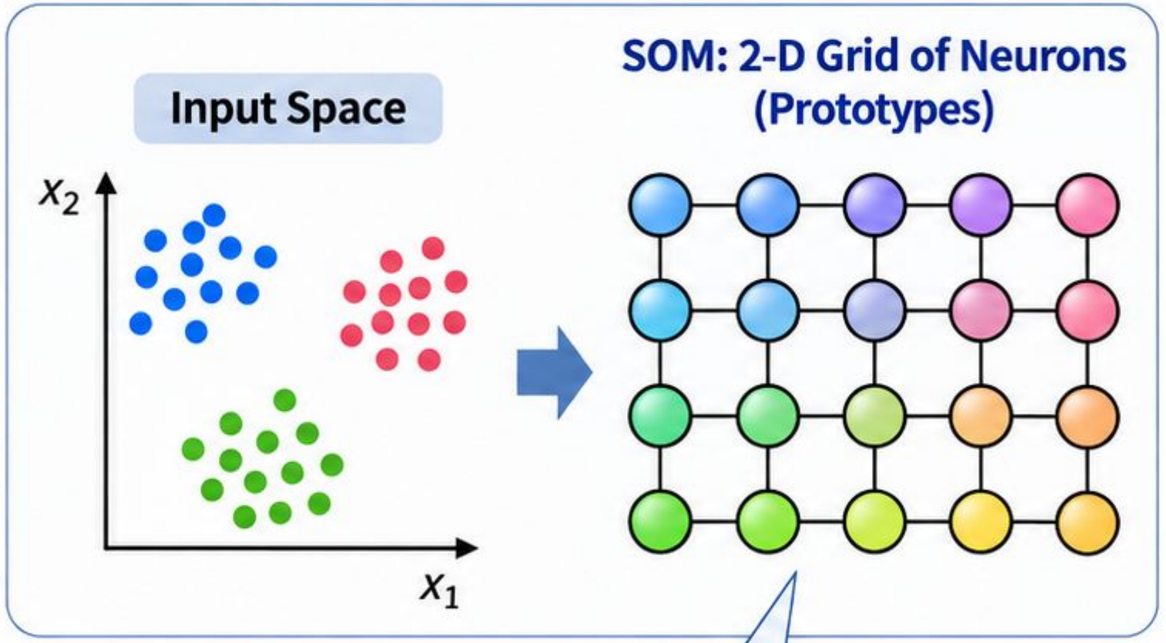
- Higher computational cost ($\approx O(n^2)$ or $O(n^3)$).
- Results can be sensitive to the choice of linkage (distance criteria).

Outline

- Introduction to Clustering
- K-Means
  - K-Means Algorithm
  - Limitation of K-Means
  - K-Means Implementation
- Agglomerative Clustering
- Self-Organizing Maps (SOM)

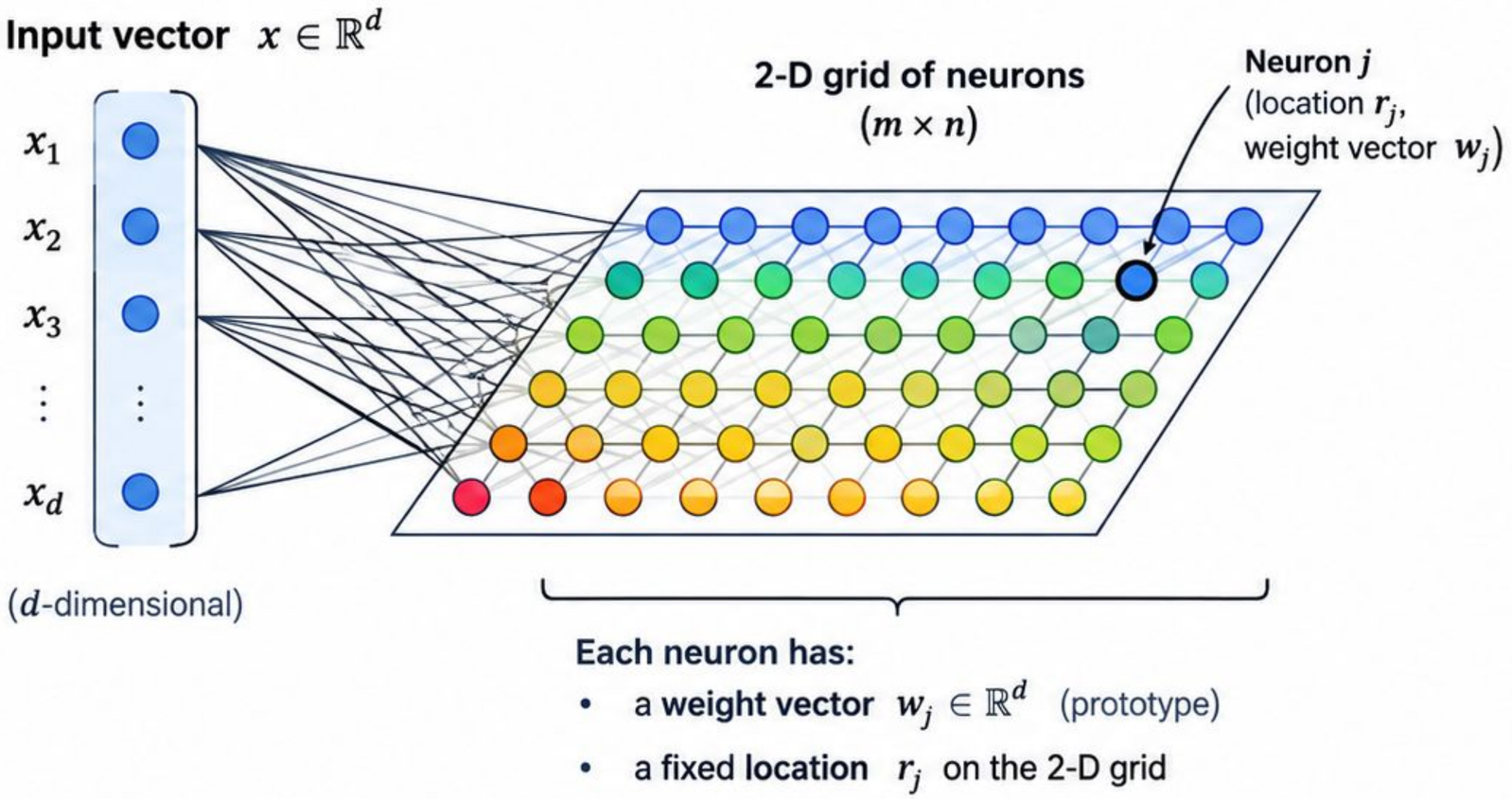
Neural Network Solution: Self-Organizing Map

- **Self-Organizing Map (SOM)** is a neural-network-based unsupervised learning method for clustering and visualization.
- **Key idea:**
  - Use a 2-D grid of neurons, each with a weight vector (prototype).
  - Learn from unlabeled data using **competitive learning.**
  - Encourage nearby neurons to represent similar patterns (**topology preservation**).
- **Uses:** clustering, data exploration, and high-dimensional data visualization.



Takeaway: SOM learns a set of prototypes and organizes them on a 2-D grid so that similar data are mapped to nearby neurons.

SOM Architecture



Key Components

1. **Input vector** $x \in \mathbb{R}^d$
   (e.g., pixel values, features)
2. **Neurons** arranged in a **2-D grid**
   $(m \times n)$ with locations r_j
3. **Each neuron** has a **weight vector**
   $w_j \in \mathbb{R}^d$ (prototype)

Key Idea

- Neurons have meaningful locations on the grid (unlike standard ANNs).
- Nearby neurons tend to represent similar patterns.

Step 1: Find the Best Matching Unit (BMU)

Goal

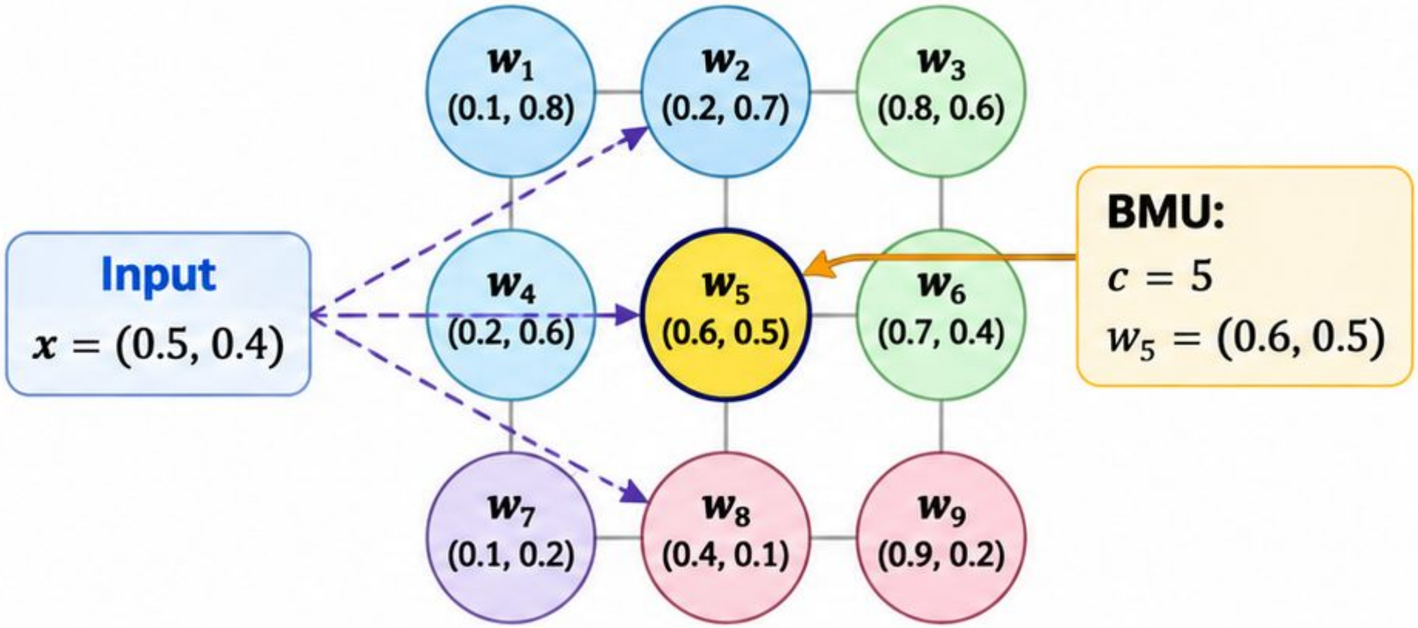
- For a given input vector $\boldsymbol{x}$, find the neuron whose weight vector is closest to $\boldsymbol{x}$.

BMU (Best Matching Unit)

$$c = \arg\min_j \|\boldsymbol{x} - \boldsymbol{w}_j\|$$

- c : index of the BMU (the "winning" neuron)
- w_j : weight vector of neuron j

Example: Find the BMU on a 2-D SOM (3 × 3)



Connection to K-Means

- This step is similar to the **assignment step** in K-Means.
- Both find the closest (prototype/centroid) to the input.

Distance to the input:

- $\|\boldsymbol{x} - \boldsymbol{w}_2\| = \|(0.5, 0.4) - (0.2, 0.7)\| = 0.424$
- $\|\boldsymbol{x} - \boldsymbol{w}_5\| = \|(0.5, 0.4) - (0.6, 0.5)\| = 0.141$ ← smallest
- $\|\boldsymbol{x} - \boldsymbol{w}_8\| = \|(0.5, 0.4) - (0.4, 0.1)\| = 0.316$

Step 2: Update the Winner and Its Neighbors

Update Rule

- Update the weight vector of the BMU and its neighbors:

$$\boldsymbol{w}_j(t+1) = \boldsymbol{w}_j(t) + \alpha(t)\, h_{cj}(t) \left[\boldsymbol{x} - \boldsymbol{w}_j(t)\right]$$

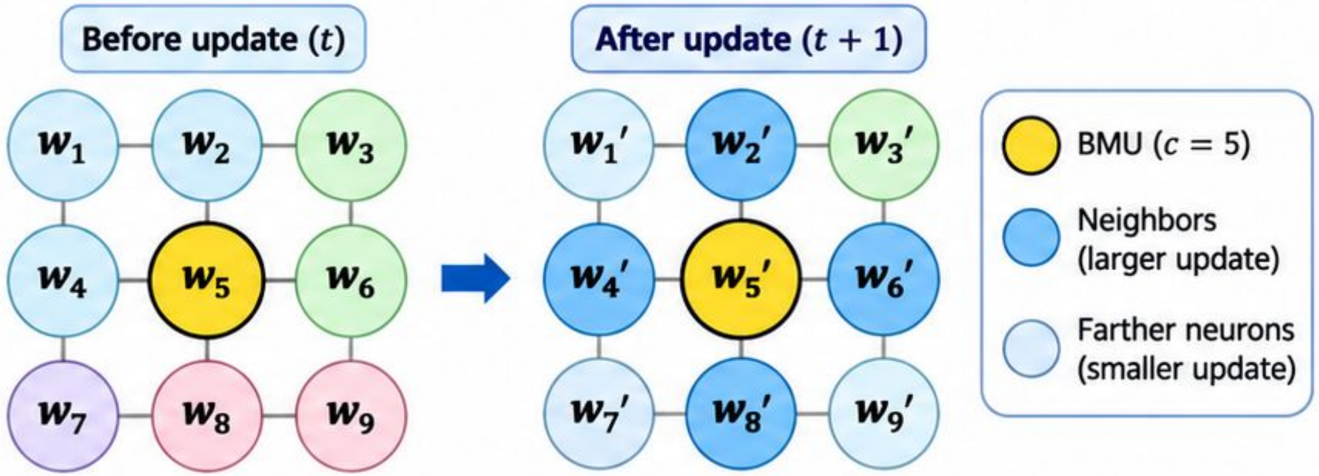
- x : input vector
- c : index of the BMU
- $\alpha(t)$: learning rate (decreases over time)
- $h_{cj}(t)$: neighborhood function

Neighborhood Function (Gaussian)

$$h_{cj}(t) = \exp\left(-\frac{\|\boldsymbol{r}_c - \boldsymbol{r}_j\|^2}{2\sigma^2(t)}\right)$$

- r_j : grid location of neuron j
- $\sigma(t)$: neighborhood radius (decreases over time)

Example: Update on a 2-D SOM (3 × 3)



- The BMU (w_5) moves the most toward x.
- Its neighbors (w_2, w_4, w_6, w_8) also move toward x, but less.
- The farther neurons (w_1, w_3, w_7, w_9) change only slightly.

Comparison to K-Means

- K-Means: update only the winning centroid.
- SOM: update **the winner and its neighboring neurons** (using a neighborhood function).

SOM Training Algorithm

1 Initialize

- Set the SOM grid size ($m \times n$) and input dimension $\boldsymbol{d}$.
- Initialize weight vectors $\boldsymbol{w_j}$ (e.g., random).
- Set initial learning rate $\alpha(0)$ and neighborhood radius $\sigma(0)$.

2 Select an Input Vector

- Choose a training sample $\boldsymbol{x}$ from the dataset.

3 Find the Best Matching Unit (BMU)

- Compute the distance between $\boldsymbol{x}$ and all weight vectors.
- Find the neuron with the minimum distance (BMU).

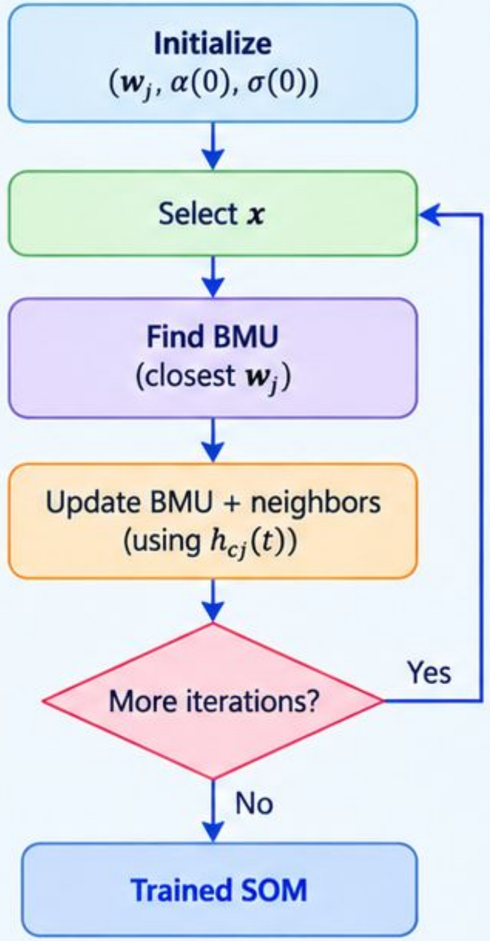
4 Update the BMU and Its Neighbors

- Adjust the weight vectors of the BMU and nearby neurons using the neighborhood function $h_{cj}(t)$.

5 Repeat

- Decrease $\alpha(t)$ and $\sigma(t)$ over time.
- Repeat steps 2–4 for all training samples and iterations.

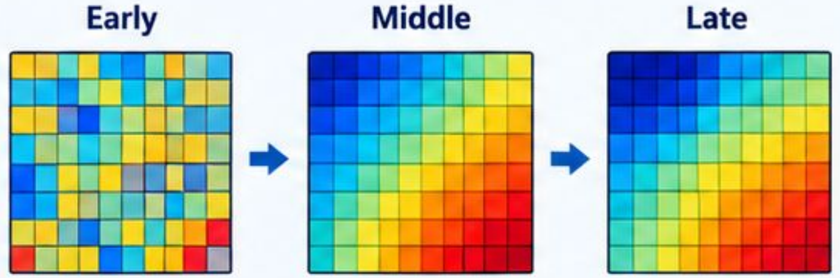
Training Flow



Key Points

- The learning rate $\alpha(t)$ and neighborhood radius $\sigma(t)$ usually decrease over time.
- The neighborhood function (e.g., Gaussian) makes nearby neurons learn similar patterns.
- SOM preserves the **topology** of the input space (nearby neurons represent similar patterns).

Behavior During Training (Example)



The map gradually organizes, and nearby neurons represent similar input patterns.

K-Means vs. SOM / SOM Visualization

K-Means	SOM
Centroids	Neuron prototypes
Closest centroid wins	BMU wins
Update winning centroid	Update BMU + neighbors
No spatial ordering	2-D topology among neurons

SOM is similar to K-Means, but it also preserves neighborhood relationships.

Summary

K-Means

- Simple and efficient prototype-based clustering
- Iteratively assigns samples and updates centroids
- Sensitive to initialization
- Works best for roughly spherical, similarly sized clusters

Agglomerative Clustering

- Bottom-up hierarchical clustering
- Repeatedly merges the most similar clusters
- Produces a dendrogram
- Can be computationally expensive for large datasets

Self-Organizing Maps (SOM)

- Neural-network-based unsupervised learning
- Uses neuron prototypes arranged on a 2-D grid
- Finds a Best Matching Unit (BMU) for each input
- Updates the BMU and its neighbors
- Preserves neighborhood relationships among similar patterns